

UCLM — Universidad de Castilla-La Mancha



Escuela Superior de Informática

Desarrollo e Integración de Servicios de IA

GuardianAI: Detección Inteligente de Fraude en Transacciones en Tiempo Real

A Data-Driven and Explainable Approach

Autores:

David Miguel Miranda Rodríguez
Alejandro Córcoles Roldán
Gonzalo Torres Aparicio
Antonio César García Fuentes

Índice general

1. Hito 1: Definición del problema y determinación del alcance	3
1.1. Contexto y definición del problema	3
1.2. Objetivos generales y específicos	4
1.3. Determinación del alcance	5
1.3.1. Planificación	5
1.3.2. Recursos Humanos: asignación de tareas y capacidad	5
1.3.3. Viabilidad. Estudio del ROI	5
1.3.4. Valor	6
2. Hito 2: Los datos: origen, descripción y análisis	7
2.1. Introducción	7
2.2. Fuentes, origen y tamaño de los datos	7
2.3. Inventario de datos	8
2.3.1. Diccionario de variables	8
2.3.2. Limitaciones del dataset	10
2.4. Análisis de los datos	11
2.4.1. Distribución de la variable objetivo	11
2.4.2. Análisis de distribuciones y correlaciones	11
2.4.3. Análisis por clase: fraude vs. no fraude	12
2.4.4. Detección de valores atípicos	15
2.4.5. Análisis de sesgos	15
2.5. Evaluación del rendimiento. Métricas	17
2.5.1. ¿Por qué no utilizar la exactitud (Accuracy)?	17
2.5.2. Métricas seleccionadas	17
2.5.3. Modelo baseline: resultados preliminares	18
2.6. Conclusiones del análisis	20
3. Hito 3: Modelado	21
3.1. Introducción	21
3.2. Selección de modelos	21
3.3. Comparativa de modelos	22
3.3.1. Análisis de Métricas Globales	23
3.3.2. Curvas de Rendimiento y Capacidad Discriminatoria	24
3.3.3. Evaluación de Errores: Matrices de Confusión	24
3.3.4. Perfil Operativo y Eficiencia del Sistema	25
3.4. Selección del modelo	25
3.4.1. Superioridad en el Manejo de la Asimetría de Clases	26
3.4.2. Eficiencia en el Frente de Pareto: Calidad vs. Tiempo	26
3.4.3. Discriminación Robusta y Curvas de Rendimiento	27
3.4.4. Resumen	27
3.5. Ajuste de hiperparámetros	27
3.5.1. Metodología de Búsqueda	28
3.5.2. Resultados de la Optimización	29

3.5.3.	Análisis de Confianza predictiva	31
3.5.4.	Cambios en Importancia de Características	31
3.6.	Resultados Finales del Modelado	32
3.7.	Análisis de explicabilidad del modelo con SHAP	34
3.7.1.	Justificación del uso de SHAP	34
3.7.2.	Interpretación global: Summary Plot	34
3.7.3.	Interpretación local: Waterfall Plot	37
3.7.4.	Conclusión del análisis de explicabilidad	38
3.8.	Conclusiones del Hito 3	38
4.	Hito 4: Despliegue del modelo y paso a producción	40
4.1.	Análisis de las necesidades del sistema	40
4.2.	Selección de tecnologías para el despliegue	41
4.3.	Del notebook al sistema: división en módulos	42
4.4.	Arquitectura de contenedores	43
4.5.	Compartición de datos entre fases	44
4.6.	API de inferencia: contrato y endpoints	45
4.7.	Modos de entrenamiento: quick y full	46
4.8.	Flujo de uso <i>plug-and-play</i>	46
4.9.	Limitaciones reconocidas y siguientes pasos	47
4.10.	Resumen	48
5.	Hito 5: Monitorización y <i>feedback loop</i>	49
5.1.	Introducción	49
5.2.	Selección de tecnologías	49
5.3.	Arquitectura de observabilidad	50
5.4.	Métricas operativas	51
5.5.	Detección de <i>data drift</i>	51
5.6.	Métricas del modelo y <i>ground truth</i> diferido	52
5.7.	Sistema de alertas y deduplicación	52
5.8.	Simulación de entorno anómalo	53
5.9.	<i>Feedback loop</i> : reentrenamiento y recarga en caliente	53
5.10.	Limitaciones reconocidas y siguientes pasos	54
5.11.	Resumen	54

Capítulo 1

Hito 1: Definición del problema y determinación del alcance

Nota sobre el contexto del proyecto: Dado el carácter estrictamente confidencial de los datos bancarios reales, este proyecto se articula en torno a un caso de uso simulado. “NovaBank” actúa como un gemelo digital (Digital Twin) representativo de una entidad financiera europea de tamaño medio. Las problemáticas de abandono de clientes y costes operativos atribuidas a NovaBank en este documento reflejan los promedios y desafíos reales del sector extraídos de la literatura científica y del conjunto de datos validado utilizado en este trabajo.

En este capítulo se detalla la definición del problema seleccionado para el desarrollo del proyecto **GuardianAI**, así como la determinación de su alcance técnico, organizativo y económico.

1.1. Contexto y definición del problema

En el ecosistema de la banca digital contemporánea, la seguridad en las transacciones con tarjeta de crédito no es solo una necesidad técnica, sino un pilar fundamental de la confianza del usuario. El fraude financiero genera pérdidas millonarias anuales (estimadas en más de 30.000 millones de dólares a nivel global según el Nilson Report [1]) que impactan directamente en la cuenta de resultados de las entidades y erosionan la reputación del sector.

Históricamente, la detección de anomalías en entidades bancarias se ha basado en sistemas de reglas estáticas [2] (por ejemplo, bloquear transacciones superiores a cierta cantidad en el extranjero). Estos enfoques presentan limitaciones evidentes: son inflexibles, no capturan patrones de comportamiento complejos y, lo más crítico, generan un volumen inaceptable de *falsos positivos*. Bloquear la tarjeta de un cliente legítimo genera un alto nivel de frustración y fricción, siendo una de las principales causas de abandono en entidades digitales como **NovaBank**.

El problema que se aborda en este proyecto consiste en utilizar datos históricos de aperturas de cuenta para generar un sistema predictivo que evalúe el riesgo de fraude en tiempo real. Para ello, se utilizará el *Bank Account Fraud Dataset Suite* [4], publicado en NeurIPS 2022, compuesto por 30 variables predictoras numéricas y categóricas (tipo de pago, situación laboral, velocidades de actividad, antigüedad del cliente, etc.), una variable temporal (`month`) y la variable objetivo `fraud_bool`. El mayor reto analítico radica en el **extremo desbalanceo de clases**: los fraudes representan únicamente un $\approx 1,1\%$ de las solicitudes (ratio aproximado 1:90). La descripción detallada de este dataset, su justificación y su análisis exploratorio se abordan en el Capítulo 2.

1.2. Objetivos generales y específicos

El problema principal puede formularse de la siguiente manera: *¿Cómo diseñar un sistema de inteligencia artificial capaz de evaluar en tiempo real el riesgo de fraude en transacciones bancarias, equilibrando la detección precisa de transacciones ilícitas o fraudulentas y la minimización de bloqueos a clientes legítimos, manteniendo un alto grado de explicabilidad en sus decisiones?*

Para dar respuesta a esta pregunta, el **objetivo general** del proyecto es diseñar, implementar y desplegar **GuardianAI**, un sistema que reduzca el fraude no detectado en un 15 % (fase MVP) para NovaBank, garantizando su integración en un entorno web real y su monitorización continua. Se establece este umbral del 15 % tomando como referencia estudios de la industria que indican que la transición de sistemas basados en reglas a modelos de *Machine Learning* produce mejoras incrementales de entre el 10 % y el 20 % en las tasas de detección durante las primeras fases de despliegue [3]. Considerando una tasa base de fraude no detectado sobre el volumen total de transacciones, una reducción del 15 % representa un impacto económico altamente significativo.

Los **objetivos específicos** son los siguientes:

- Construir un *pipeline* de datos automatizado capaz de mitigar el desbalanceo extremo de clases. Inicialmente se contempla sobremuestreo sintético (SMOTE), si bien la elección final entre SMOTE, ponderación de clases (`class_weight`) o ajuste de `scale_pos_weight` en XGBoost se realizará empíricamente en el Hito 3 en función del comportamiento del modelo sobre el conjunto de test original desbalanceado.
- Implementar y entrenar un modelo predictivo evaluando candidatos basados en *Extreme Gradient Boosting* (XGBoost). La elección preliminar de esta familia de modelos se justifica por su rendimiento estado-del-arte documentado en la literatura científica para conjuntos de datos tabulares, altamente desbalanceados y no lineales, características propias de las transacciones financieras [3].
- Priorizar métricas técnicas de *Machine Learning* adecuadas para datos severamente desbalanceados, como el Área bajo la curva Precision-Recall (AUPRC) y el F_1 -Score, descartando la exactitud (*Accuracy*) como métrica principal, y alineando estas métricas posteriormente con el coste de negocio.
- Integrar una capa de explicabilidad mediante la librería SHAP (*SHapley Additive exPlanations*), permitiendo justificar de forma interpretable el motivo del bloqueo de una operación.
- Empaquetar el modelo como un microservicio mediante FastAPI y Docker.
- Diseñar un sistema de monitorización capaz de detectar *Data Drift*, simulando ataques de fraude emergentes como el *Carding* (micro-fraudes iterativos).
- Asegurar el sistema desde el diseño, para evitar futuras fugas de datos.

1.3. Determinación del alcance

1.3.1. Planificación

El proyecto se desarrollará a lo largo de 12 semanas, estructurado en 5 grandes hitos iterativos: definición del problema, ingeniería de datos, modelado, despliegue en API y monitorización de drift en producción.

1.3.2. Recursos Humanos: asignación de tareas y capacidad

El desarrollo de GuardianAI refleja un escenario realista de producto de IA, donde interactúan perfiles de modelado, ingeniería de datos, operaciones y negocio. En nuestro entorno simulado, el equipo asume los roles técnicos y funcionales necesarios para interactuar con los requisitos de NovaBank, actuando de forma dual como *stakeholders* y desarrolladores del sistema.

El equipo técnico está formado por cuatro ingenieros junior con roles específicos:

David Miguel Miranda Rodríguez. Rol principal: Analista / Ingeniero de Datos. Responsable del diseño del *pipeline* de ingesta y preprocesamiento. Su labor incluye la limpieza de los datos tabulares, el escalado estandarizado de variables numéricas, la codificación de variables categóricas y la evaluación empírica de técnicas de manejo del desbalance de clases (*resampling* SMOTE frente a ponderación por clases) para seleccionar la alternativa que produzca mejor desempeño sobre datos de test representativos.

Alejandro Córcoles Roldán. Rol principal: Machine Learning Engineer. Encargado del núcleo algorítmico del sistema. Sus tareas abarcan la selección del modelo (XGBoost), la optimización de hiperparámetros y la validación cruzada. Su principal desafío es garantizar que el modelo no caiga en la “trampa del Accuracy” y logre maximizar la identificación de fraudes (Recall) sin penalizar severamente las transacciones legítimas (Precision).

Gonzalo Torres Aparicio. Rol principal: Analista de Negocio y Explicabilidad (Domain Expert). Ejerce de puente entre el modelo matemático y los requisitos simulados de NovaBank. Es el responsable de implementar SHAP para desgranar la caja negra del modelo de XGBoost. Su labor asegura que si un cliente reclama un bloqueo, el banco pueda darle una respuesta basada en datos (ej. “*Comportamiento atípico combinado con un monto inusual para la franja horaria*”).

Antonio César García Fuentes. Rol principal: DevOps y MLOps Engineer. Responsable de sacar el modelo del entorno de desarrollo (*notebooks*) y llevarlo a producción. Sus tareas incluyen el empaquetado del sistema en contenedores Docker, el desarrollo de la API con FastAPI y la configuración de *dashboards* para monitorizar la caída de rendimiento ante cambios de comportamiento de los estafadores (*Data Drift*).

1.3.3. Viabilidad. Estudio del ROI

Para evaluar la viabilidad de GuardianAI, se realiza una estimación del Retorno de Inversión (ROI) considerando un escenario de 12 semanas de desarrollo. Los costes se han presupuestado basándose en los estándares actuales del mercado tecnológico:

- **Horas totales:** $4 \text{ personas} \times 10 \text{ horas/semana} \times 12 \text{ semanas} = 480 \text{ horas}$.
- **Coste humano:** $480 \text{ horas} \times 35 \text{ €/hora} = 16.800 \text{ €}$. (Tarifa estimada para perfiles Junior/Mid en Ciencia de Datos en España).

- **Coste de infraestructura (Cloud):** 1.500 €. La externalización de la infraestructura es estrictamente necesaria debido a que la fase de modelado requerirá el entrenamiento iterativo de algoritmos avanzados de *Machine Learning*, sumado a la probable generación de datos sintéticos para mitigar el extremo desbalanceo de clases y una exhaustiva búsqueda de hiperparámetros. Estas operaciones de experimentación exigen picos de memoria RAM y capacidad de procesamiento (CPU/GPU) que exceden las prestaciones de los equipos locales de desarrollo. El presupuesto se calcula en base a un consumo estimado de 500 € mensuales durante los 3 meses de desarrollo (fases de experimentación y MVP), desglosado de la siguiente manera:
 - *Cómputo para experimentación:* ~300 €/mes (Instancias Cloud orientadas a cómputo intensivo para la validación y el entrenamiento cruzado de diferentes familias de modelos predictivos).
 - *Almacenamiento y registro:* ~50 €/mes (Servicios tipo S3 para alojar los *datasets* crudos y procesados, así como el registro de los experimentos).
 - *Despliegue del MVP:* ~150 €/mes (Alojamiento de contenedores y ejecución de la API para la simulación de inferencia de fraude en tiempo real).
- **Coste Total = 18.300 €.**

Se estima que, al lograr el MVP (reducción del 15 % en fraudes exitosos) y ahorrar más de 200 horas mensuales en auditorías manuales, NovaBank obtendrá un beneficio semestral de 26.000 €. El ROI se calcula mediante la siguiente fórmula:

$$ROI = \frac{26000 - 18300}{18300} = 0,42 \quad (1.1)$$

Un ROI del 42 % en la fase inicial demuestra que el proyecto es financieramente viable. Adicionalmente, para garantizar que el proyecto no incurra en pérdidas (Punto de Equilibrio o *Break-Even*), el ahorro generado debe igualar la inversión de 18.300 €. Asumiendo una relación lineal entre el porcentaje de detección y el ahorro, la reducción mínima requerida de fraude no detectado para hacer viable el proyecto es del 10,55 % ($18,300 / (26,000 / 0,15)$). Alcanzar el objetivo propuesto del 15 % sitúa al proyecto con un amplio margen de seguridad por encima de este límite inferior.

1.3.4. Valor

La implementación de GuardianAI aporta un valor multidimensional a NovaBank:

1. **Valor Económico y Organizativo:** Genera ahorros directos por reducción de contracargos y optimiza el tiempo de los analistas humanos, quienes solo revisarán las alertas marcadas como dudosas por la IA.
2. **Valor Social y de Usuario:** Reduce la fricción con los clientes honestos al minimizar los falsos positivos. Un cliente no frustrado es un cliente fidelizado.
3. **Valor Tecnológico y Estratégico:** Introduce la cultura MLOps en el banco. Al disponer de una infraestructura que monitoriza el *Data Drift*, la entidad no solo reacciona al fraude actual, sino que se prepara tecnológicamente para readaptarse a ataques futuros.

Capítulo 2

Hito 2: Los datos: origen, descripción y análisis

2.1. Introducción

En este segundo hito se describen y analizan los **datos** de los que se dispone para abordar el proyecto descrito en el capítulo anterior, consistente en diseñar, implementar y desplegar un sistema inteligente de detección de fraude en aperturas de cuentas bancarias para NovaBank. El sistema desarrollado debe ser capaz de asignar una **puntuación de riesgo** (*fraud risk scoring*) a cada solicitud [5], permitiendo a los analistas priorizar la revisión de las operaciones más sospechosas.

Como referencia de implementación y para facilitar la reproducibilidad, el código desarrollado en GuardianAI está disponible en el repositorio oficial del proyecto en GitHub¹.

El presente capítulo se organiza de la siguiente manera: en primer lugar, se describen las fuentes, el origen y el tamaño del conjunto de datos utilizado (§2.2). A continuación, se realiza un inventario detallado de las variables disponibles, incluyendo sus tipos, descripciones y las limitaciones identificadas (§2.3). Posteriormente, se presenta un análisis exploratorio exhaustivo de los datos (§2.4), cuyas visualizaciones han sido generadas en el cuaderno de Jupyter asociado a este hito. Finalmente, se discuten las métricas de evaluación seleccionadas para medir el rendimiento del sistema (§2.5) y se resumen las conclusiones del análisis (§2.6).

2.2. Fuentes, origen y tamaño de los datos

Los datos utilizados en este proyecto provienen del **Bank Account Fraud Dataset Suite**, un conjunto de datos publicado en la conferencia NeurIPS 2022 por Jesus et al. [4] y disponible públicamente en la plataforma Kaggle². Este dataset fue diseñado específicamente para la investigación en detección de fraude financiero, abordando las carencias de los conjuntos de datos tradicionales en este dominio.

Justificación de la elección del dataset

La selección de este dataset se fundamenta en varias razones:

- **Validación académica:** Al haber sido presentado en NeurIPS, el dataset ha sido revisado por la comunidad científica y cumple con estándares de calidad y reproducibilidad.

¹<https://github.com/davidmir39/GuardianAI>

²<https://www.kaggle.com/datasets/sgpjesus/bank-account-fraud-dataset-neurips-2022>

- **Realismo controlado:** Aunque los datos son sintéticos, fueron generados a partir de patrones reales de fraude bancario, lo que garantiza que las distribuciones, correlaciones y patrones de comportamiento reflejan escenarios verosímiles.
- **Privacidad:** Al tratarse de datos simulados, no existe riesgo de exposición de información personal, lo que permite un análisis y publicación sin restricciones éticas ni legales.
- **Desafíos realistas:** El dataset incorpora intencionadamente desbalanceo de clases, ruido en los datos y variables con distribuciones complejas, características habituales en los datos de fraude del mundo real.

Dado que NovaBank es un gemelo digital representativo de una entidad financiera europea (véase la nota del Capítulo 1), este dataset constituye una fuente de datos coherente con el escenario simulado del proyecto.

Características generales del dataset

El conjunto de datos presenta las siguientes características:

- **Número de registros:** 1.000.000 de solicitudes de apertura de cuenta.
- **Número de variables:** 32 columnas (30 variables predictoras + 1 variable objetivo + 1 variable temporal).
- **Formato:** Datos tabulares estructurados en formato CSV.
- **Tipos de variables:** Combinación de variables numéricas continuas, discretas enteras, binarias y categóricas.
- **Variable objetivo:** `fraud_bool` (binaria: 0 = legítima, 1 = fraudulenta).

Se trata, por tanto, de un problema de **datos tabulares estructurados** con un volumen considerable (1 millón de registros), lo que permite aplicar técnicas de aprendizaje automático supervisado sin restricciones de tamaño muestral.

2.3. Inventario de datos

En esta sección se define el conjunto de datos con el que se va a trabajar para el desarrollo del sistema. Se presenta un diccionario de variables y se identifican las limitaciones del dataset que condicionarán las decisiones de modelado.

2.3.1. Diccionario de variables

La Tabla 2.1 recoge las 32 variables del dataset, clasificadas por tipo y con una breve descripción de su significado en el contexto de la detección de fraude en aperturas de cuentas bancarias.

Tabla 2.1: Inventario de datos: diccionario de variables del dataset.

Variable	Tipo	Descripción
fraud_bool	Binaria	Variable objetivo: 1 = fraude, 0 = legítima.
income	Continua	Ingresos anuales declarados por el solicitante.
name_email_similarity	Continua	Similitud entre el nombre del solicitante y su email.
prev_address_months_count	Entera	Meses en la dirección anterior (-1 = no disponible).
current_address_months_count	Entera	Meses en la dirección actual (-1 = no disponible).
customer_age	Entera	Rango de edad del solicitante (10–90 en intervalos de 10).
days_since_request	Continua	Días transcurridos desde la solicitud.
intended_balcon_amount	Continua	Importe previsto de la primera operación con la cuenta.
payment_type	Categorica	Tipo de pago utilizado en la solicitud.
zip_count_4w	Entera	Solicitudes desde el mismo código postal (4 semanas).
velocity_6h	Continua	Velocidad de transacciones en las últimas 6 horas.
velocity_24h	Continua	Velocidad de transacciones en las últimas 24 horas.
velocity_4w	Continua	Velocidad de transacciones en las últimas 4 semanas.
bank_branch_count_8w	Entera	Solicitudes en la misma sucursal (8 semanas).
date_of_birth_distinct_emails_4w	Entera	Emails distintos con la misma fecha de nacimiento.
employment_status	Categorica	Situación laboral del solicitante.
credit_risk_score	Entera	Puntuación de riesgo crediticio (eliminada).
email_is_free	Binaria	Si el email proviene de un proveedor gratuito.
housing_status	Categorica	Tipo de vivienda del solicitante.
phone_home_valid	Binaria	Si el teléfono fijo es válido.
phone_mobile_valid	Binaria	Si el teléfono móvil es válido.
bank_months_count	Entera	Meses como cliente del banco (-1 = no disponible).
has_other_cards	Binaria	Si el solicitante posee otras tarjetas.
proposed_credit_limit	Continua	Límite de crédito propuesto.
foreign_request	Binaria	Si la solicitud proviene de una IP extranjera.
source	Categorica	Canal de origen de la solicitud (web, app...).
session_length_in_minutes	Continua	Duración de la sesión (-1 = no disponible).
device_os	Categorica	Sistema operativo del dispositivo.
keep_alive_session	Binaria	Si la sesión se mantuvo activa.
device_distinct_emails_8w	Entera	Emails distintos desde el mismo dispositivo (8 sem.).
device_fraud_count	Entera	Fraudes conocidos asociados al dispositivo.
month	Entera	Mes de la solicitud (0–7, abarcando 8 meses).

Decisión de diseño: eliminación de `credit_risk_score`

Durante la fase de inspección inicial se detectó que la variable `credit_risk_score` presenta una correlación extremadamente alta con la variable objetivo. Tras analizar su naturaleza, se determinó que esta variable constituye una **fuga de datos** (*data leakage*): se trata de una puntuación calculada a posteriori que incorpora información sobre el resultado de la solicitud. Incluirla en el modelo generaría resultados artificialmente optimistas que no serían reproducibles en un entorno de producción real. Por este motivo, **se ha eliminado del dataset** antes de cualquier análisis o entrenamiento.

2.3.2. Limitaciones del dataset

Antes de comenzar con el análisis, es imprescindible identificar las limitaciones del conjunto de datos que condicionarán tanto el análisis como las decisiones de modelado:

1. **Desbalanceo extremo de clases:** La variable objetivo presenta un desbalanceo severo: aproximadamente el 1,1 % de las solicitudes son fraudulentas (11.029 casos positivos de un total de 1.000.000). Esto implica que un modelo trivial que predijera siempre “no fraude” obtendría una exactitud del 98,9 %, pero sería completamente inútil. Las métricas de evaluación deben seleccionarse cuidadosamente (véase §2.5).
2. **Naturaleza sintética de los datos:** Aunque los datos están generados siguiendo patrones reales, no dejan de ser sintéticos. Los patrones de fraude pueden no capturar toda la complejidad y variabilidad del mundo real (estacionalidad, adaptación de los estafadores, contexto socioeconómico).
3. **Valores centinela (-1):** Varias variables (`prev_address_months_count`, `current_address_months_count`, `bank_months_count`, `session_length_in_minutes`, `device_distinct_emails_8w`) utilizan el valor `-1` para indicar información no disponible. Estos valores no son nulos en sentido estricto, sino marcadores que deben tratarse explícitamente durante el preprocesamiento.
4. **Correlaciones moderadas con la variable objetivo:** Las correlaciones lineales individuales de cada variable con `fraud_bool` son muy bajas (ninguna supera $|r| \approx 0,07$, siendo `proposed_credit_limit`, `customer_age` y `keep_alive_session` las tres más fuertes), lo que sugiere que la detección de fraude requiere capturar interacciones entre múltiples variables, no señales aisladas.
5. **Categorías infrarrepresentadas:** Algunas categorías en variables categóricas presentan muy pocas muestras, lo que puede generar inestabilidad en las estimaciones y sobreajuste.
6. **Variabilidad temporal limitada:** Los datos abarcan únicamente 8 meses (`month`: 0–7), lo que limita la capacidad de capturar patrones estacionales anuales o tendencias a largo plazo.
7. **Variables con varianza cercana a cero:** Algunas variables, como `device_fraud_count`, presentan una concentración extrema en un único valor (típicamente cero), lo que reduce su capacidad informativa individual. No obstante, estas variables de baja varianza pueden seguir siendo relevantes en interacciones con otras características.

8. **Naturaleza estática del dataset:** Aunque el título original del dataset incluye el término “*Dynamic*”, los datos utilizados en este proyecto corresponden a una instantánea estática. No se dispone de actualizaciones incrementales ni de un flujo de datos en tiempo real que permita evaluar la adaptabilidad del modelo ante la evolución de las estrategias de los estafadores (*concept drift*).

2.4. Análisis de los datos

El análisis exploratorio de los datos se ha realizado de forma exhaustiva en el cuaderno de Jupyter `src/01_eda_and_baseline.ipynb` asociado a este hito. En esta sección se resumen los hallazgos más relevantes, referenciando las visualizaciones generadas.

2.4.1. Distribución de la variable objetivo

El punto de partida del análisis es comprender la distribución de la variable que se pretende predecir. Como muestra la Figura 2.1, el dataset presenta un **desbalanceo extremo**: del millón de solicitudes, solo un $\sim 1,1\%$ corresponden a fraude (11.029 casos positivos). Este ratio de desbalanceo (aproximadamente 1:90) condiciona todas las decisiones posteriores, desde la selección de métricas hasta las técnicas de muestreo.

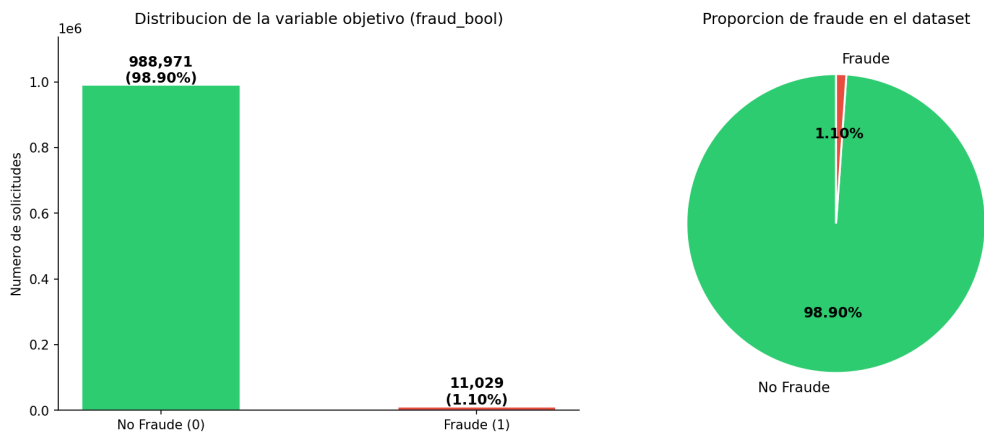


Figura 2.1: Distribución de la variable objetivo `fraud_bool`. Se observa un desbalanceo extremo: aproximadamente el 1,1 % de las solicitudes son fraudulentas.

Este desbalanceo es coherente con la realidad del sector bancario, donde las transacciones fraudulentas son una fracción mínima del total. Sin embargo, desde el punto de vista del modelado, implica que el clasificador debe ser especialmente sensible a la clase minoritaria sin generar un exceso de falsos positivos.

2.4.2. Análisis de distribuciones y correlaciones

Se ha analizado la distribución de cada variable del dataset, diferenciando entre variables continuas, discretas y categóricas. Los principales hallazgos son:

- **Variables continuas** (`income`, `name_email_similarity`, `velocity_*`, etc.): Presentan distribuciones asimétricas con colas largas hacia la derecha. Las variables de velocidad (`velocity_6h`, `velocity_24h`, `velocity_4w`) muestran una concentración significativa en valores bajos con outliers extremos.

- **Variables discretas** (`customer_age`, `zip_count_4w`, etc.): Muchas presentan distribuciones concentradas en unos pocos valores, con algunas variables casi constantes (como `device_fraud_count`, con varianza cercana a cero).
- **Variables categóricas** (`payment_type`, `housing_status`, etc.): Muestran distribuciones desiguales, con algunas categorías dominantes y otras con muy pocas observaciones.

En cuanto a las correlaciones entre variables numéricas, se ha calculado la matriz de correlación de Pearson y la asociación entre variables categóricas mediante el coeficiente V de Cramér. Los resultados muestran que:

- Las correlaciones más altas entre predictores se dan entre las variables de velocidad y el tiempo: `velocity_4w` y `month` ($r \approx -0,85$), `velocity_24h` y `month` ($r \approx -0,55$), y entre `velocity_24h` y `velocity_4w` ($r \approx 0,54$). Es esperable, dado que miden fenómenos de actividad en ventanas temporales solapadas.
- No existe ninguna variable individualmente muy correlacionada con `fraud_bool`: las correlaciones más altas (`proposed_credit_limit`, `customer_age`, `keep_alive_session`) apenas alcanzan $|r| \approx 0,05-0,07$, confirmando que la detección de fraude es un problema intrínsecamente multivariante.

2.4.3. Análisis por clase: fraude vs. no fraude

Para evaluar la capacidad discriminativa de las variables, se han generado histogramas de densidad y diagramas de caja (boxplots) separando las solicitudes por clase (fraude vs. no fraude) para las 10 variables más correlacionadas con la variable objetivo.

Los histogramas de la Figura 2.2 y los boxplots de la Figura 2.3 revelan patrones relevantes:

- Variables económicas (`proposed_credit_limit`, `income`) y demográficas (`customer_age`): las solicitudes fraudulentas se concentran en franjas específicas, con valores medios superiores de `proposed_credit_limit` y una distribución de `customer_age` desplazada respecto a las legítimas.
- La variable `name_email_similarity` presenta diferencias sutiles pero consistentes: los fraudes tienden a mostrar valores de similitud ligeramente más bajos, sugiriendo el uso de correos electrónicos no relacionados con el nombre real.
- Variables de identidad e interacción (`has_other_cards`, `phone_home_valid`, `keep_alive_session`, `device_distinct_emails_8w`): se aprecian diferencias de proporción marcadas entre clases, coherentes con perfiles de dispositivo reutilizados para ataques.
- Las variables de antigüedad de dirección (`current_address_months_count`) muestran que las solicitudes fraudulentas proceden con mayor frecuencia de usuarios con historial corto en su domicilio actual.

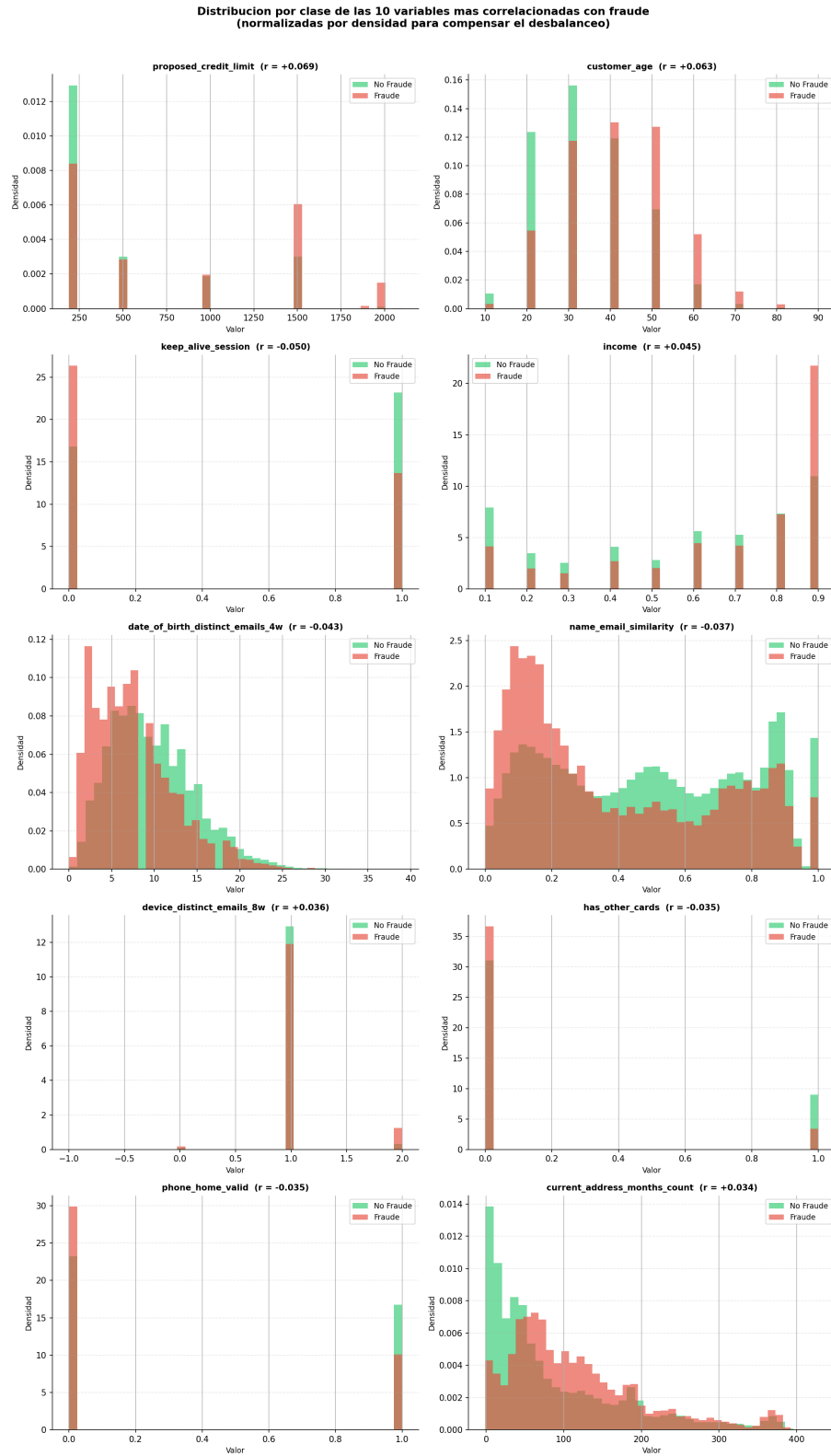


Figura 2.2: Histogramas por clase (densidad normalizada) para las 10 variables más correlacionadas con fraude. La normalización por densidad permite comparar las distribuciones a pesar del desbalanceo.

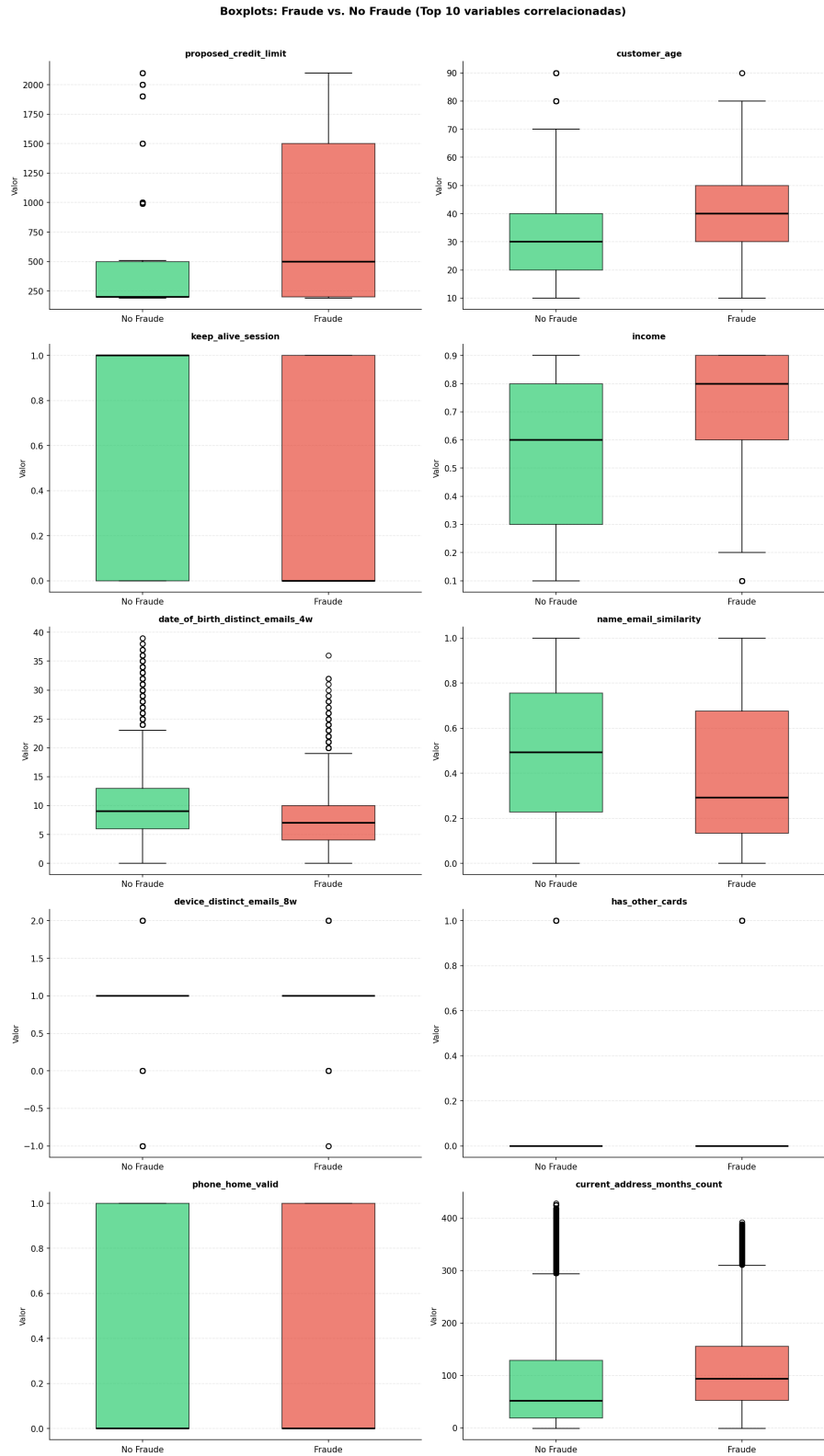


Figura 2.3: Boxplots comparativos entre solicitudes legítimas y fraudulentas para las variables más relevantes.

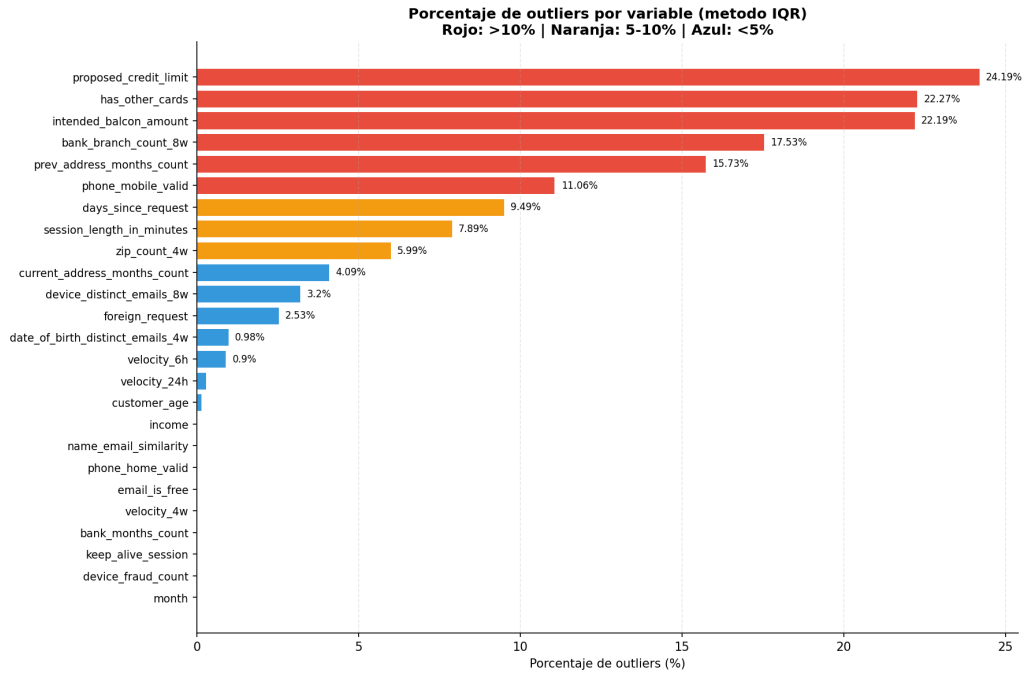


Figura 2.4: Porcentaje de outliers por variable según el método IQR. Las variables de velocidad y actividad reciente presentan los mayores porcentajes de valores atípicos.

2.4.4. Detección de valores atípicos

Se ha aplicado el método del rango intercuartílico (IQR) para cuantificar la presencia de valores atípicos en cada variable numérica. La Figura 2.4 muestra el porcentaje de outliers por variable.

Es importante destacar que, en el contexto de la detección de fraude, los valores atípicos no deben eliminarse automáticamente. Precisamente los comportamientos anómalos —solicitudes con velocidades inusuales, múltiples emails desde el mismo dispositivo o sesiones de duración extrema— son **indicadores potenciales de fraude**. El tratamiento de outliers debe ser, por tanto, selectivo y orientado al problema.

2.4.5. Análisis de sesgos

Un aspecto fundamental del análisis de datos en sistemas de IA es la identificación de sesgos que puedan afectar al modelo. Se ha analizado si la tasa de fraude varía significativamente según las variables categóricas y binarias del dataset.

Las Figuras 2.5 y 2.6 revelan varios patrones de interés:

- **Tipo de pago:** Ciertos métodos de pago presentan tasas de fraude significativamente superiores a la media, lo que sugiere que los estafadores tienen preferencia por determinados canales.
- **Situación laboral:** Existen diferencias en la tasa de fraude según el tipo de empleo, aunque deben interpretarse con cautela para evitar discriminación algorítmica.
- **Solicitudes extranjeras:** La variable `foreign_request` muestra una tasa de fraude superior para solicitudes desde IPs extranjeras, un patrón esperado en el dominio.

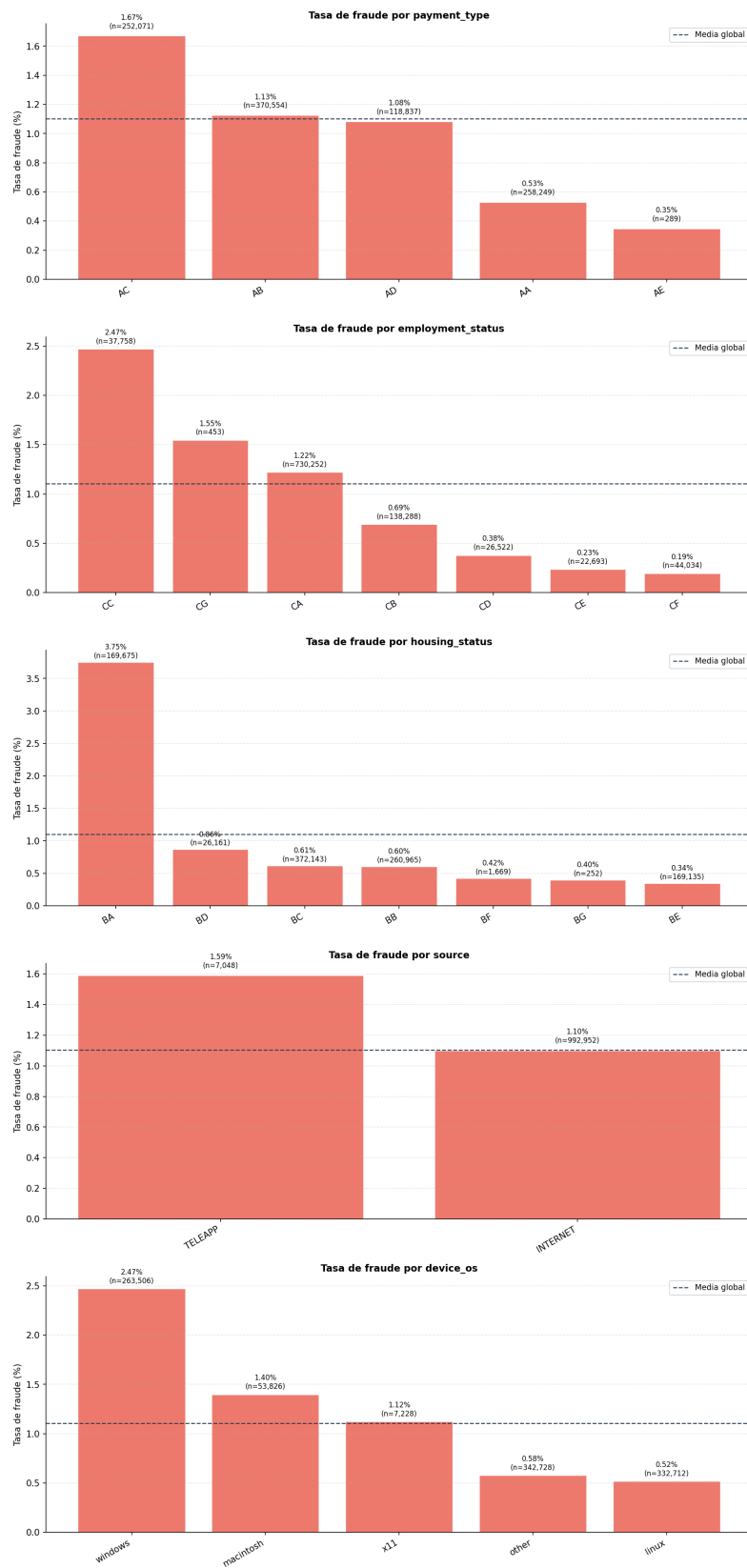


Figura 2.5: Tasa de fraude desglosada por cada variable categórica. Se observan diferencias relevantes según el tipo de pago, la situación laboral y el tipo de vivienda.

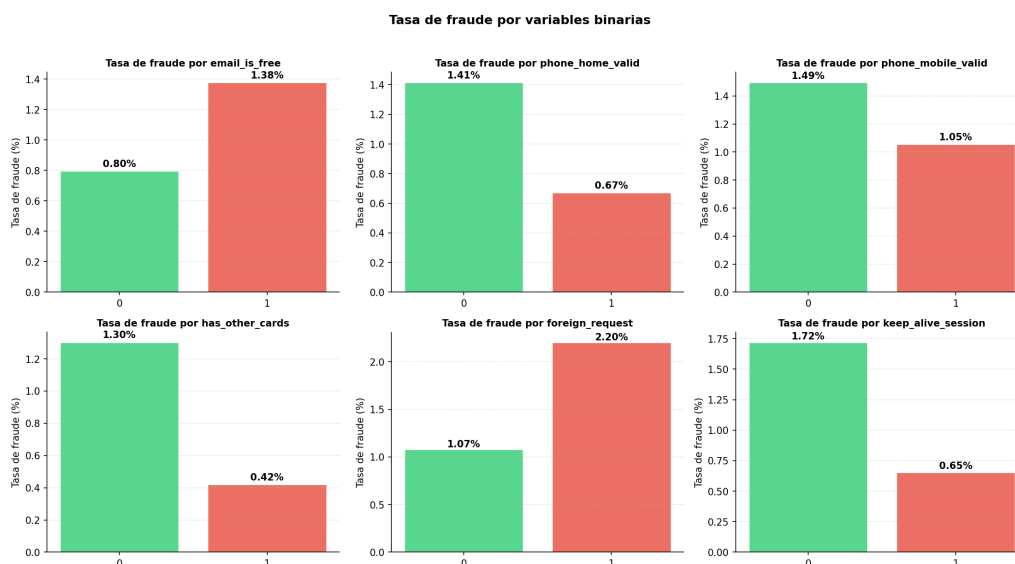


Figura 2.6: Tasa de fraude desglosada por variables binarias. Variables como `foreign_request` y `keep_alive_session` muestran diferencias pronunciadas.

- **Sesiones mantenidas activas:** Las solicitudes con `keep_alive_session` activado presentan diferencias en la tasa de fraude, lo que puede estar relacionado con el uso de bots o herramientas automatizadas.

Estos sesgos en los datos no son necesariamente errores, sino reflejos de patrones reales de comportamiento fraudulento. Sin embargo, el modelo debe diseñarse de forma que **no amplifique sesgos injustos** contra determinados grupos demográficos.

2.5. Evaluación del rendimiento. Métricas

Una vez analizados los datos, es necesario definir cómo se evaluará el rendimiento del sistema de detección de fraude. La elección de las métricas adecuadas es especialmente crítica en problemas con desbalanceo extremo de clases.

2.5.1. ¿Por qué no utilizar la exactitud (Accuracy)?

En un dataset donde aproximadamente el 98,9 % de las solicitudes son legítimas, un modelo que siempre prediga “no fraude” obtendría una exactitud del 98,9 %. Sin embargo, este modelo sería completamente inútil desde el punto de vista del negocio, ya que no detectaría ningún fraude. Por tanto, la **exactitud no es una métrica adecuada** para este problema.

2.5.2. Métricas seleccionadas

Se han seleccionado las siguientes métricas, alineadas tanto con los estándares de la comunidad científica como con los objetivos de negocio de NovaBank:

- **Precision (Precisión):** Proporción de alertas de fraude que efectivamente son fraude. Una precisión baja significa que el banco está investigando demasiadas falsas alarmas, generando costes operativos y fricción con los clientes.

- **Recall (Sensibilidad):** Proporción de fraudes reales que el modelo detecta. Un recall bajo implica que se están dejando pasar fraudes, con el consiguiente impacto económico.
- **F_1 -Score:** Media armónica de Precision y Recall. Útil como métrica resumen que penaliza el desequilibrio entre ambas.
- **PR-AUC (Área bajo la curva Precision-Recall):** Métrica especialmente recomendada para problemas con desbalanceo extremo [4], ya que evalúa el rendimiento del modelo en el espacio Precision-Recall, que es más informativo que el espacio ROC en estos contextos.
- **ROC-AUC (Área bajo la curva ROC):** Métrica complementaria que mide la capacidad del modelo para discriminar entre clases a diferentes umbrales de decisión.

2.5.3. Modelo baseline: resultados preliminares

Para establecer una línea base de rendimiento, se ha entrenado un modelo **RandomForestClassifier** con los siguientes hiperparámetros: `n_estimators=50`, `max_depth=10`, `class_weight='balanced'`. El uso de `class_weight='balanced'` permite al modelo asignar mayor peso a la clase minoritaria durante el entrenamiento.

La Figura 2.7 muestra las 20 variables más importantes según el criterio de importancia Gini del modelo.

La Figura 2.8 presenta la matriz de confusión, la curva ROC y la curva Precision-Recall del modelo baseline.

Los resultados obtenidos con este modelo baseline son los siguientes:

Tabla 2.2: Métricas del modelo baseline (RandomForest).

Métrica	Valor
Precision	0,05
Recall	0,72
F_1 -Score	0,10
PR-AUC	0,13
ROC-AUC	0,88
Accuracy	86 %

Estos resultados merecen una interpretación cuidadosa:

- El **Recall de 0,72** indica que el modelo detecta el 72 % de los fraudes reales. Esto es alentador como punto de partida, ya que demuestra que las variables del dataset contienen señal discriminativa.
- La **Precision de 0,05** es muy baja, indicando que por cada fraude detectado correctamente, el modelo genera aproximadamente 18 falsas alarmas. Esto es inasumible en producción.
- El **PR-AUC de 0,13**, aunque bajo, es significativamente superior a la tasa base de fraude ($\sim 0,011$), lo que confirma que el modelo aporta valor frente a una predicción aleatoria.

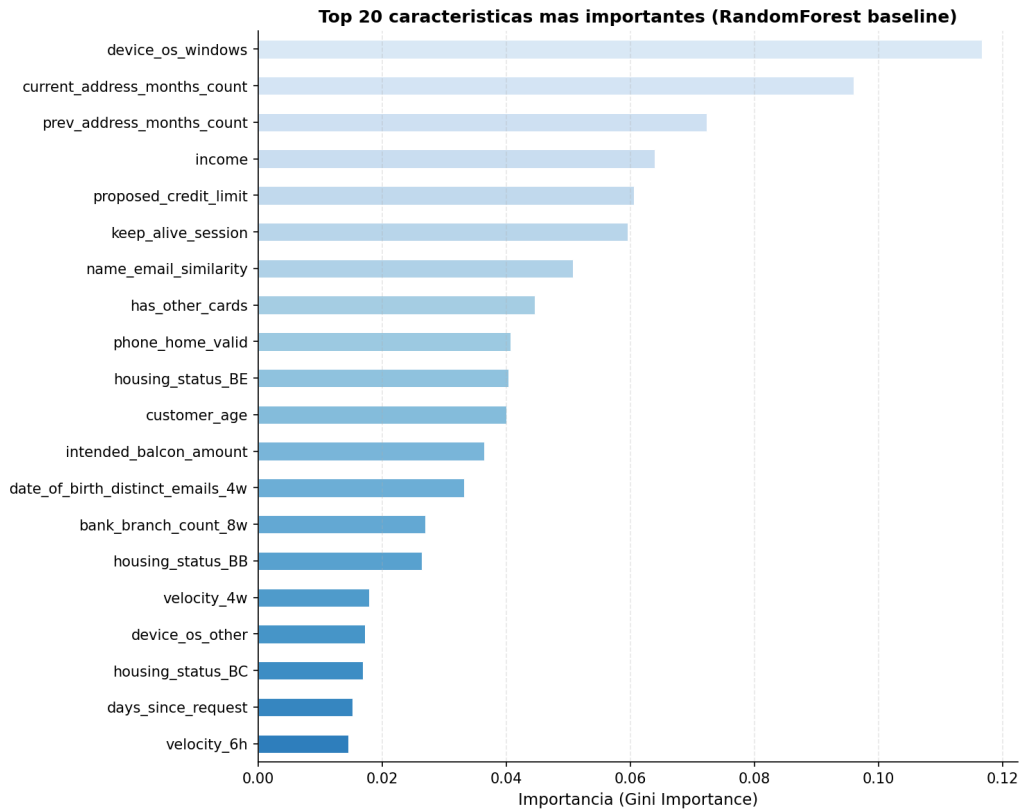


Figura 2.7: Top 20 características más importantes según la importancia Gini del RandomForest. Las 10 más informativas son `device_os_windows` (0,117), `current_address_months_count` (0,096), `prev_address_months_count` (0,072), `income` (0,064), `proposed_credit_limit` (0,061), `keep_alive_session` (0,060), `name_email_similarity` (0,051), `has_other_cards` (0,045), `phone_home_valid` (0,041) y `housing_status_BE` (0,040).

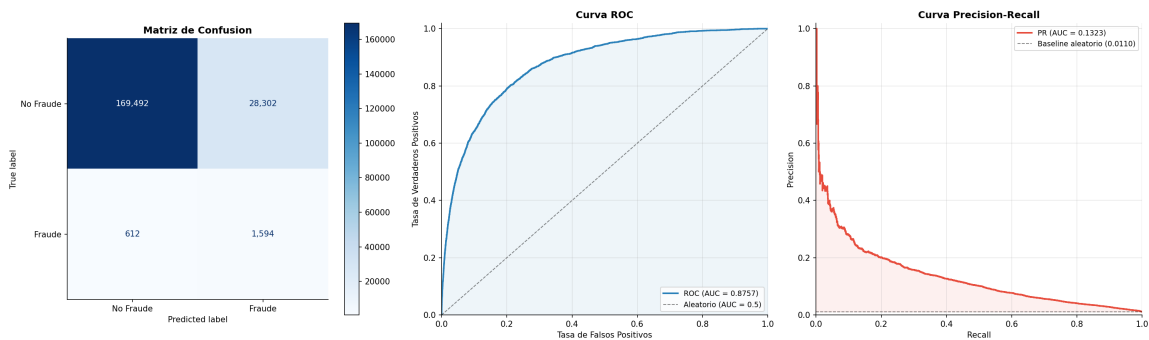


Figura 2.8: Evaluación del modelo baseline: matriz de confusión (izquierda), curva ROC (centro) y curva Precision-Recall (derecha).

Estos resultados son **preliminares** y se optimizarán en el Hito 3 mediante técnicas de ingeniería de características, selección de modelos más sofisticados (XGBoost, LightGBM), ajuste de hiperparámetros y optimización del umbral de decisión. El objetivo es reducir significativamente los falsos positivos manteniendo un recall alto, encontrando el equilibrio óptimo que minimice el coste total para NovaBank.

2.6. Conclusiones del análisis

El análisis exploratorio realizado en este hito permite extraer las siguientes conclusiones fundamentales:

1. **El dataset contiene señal discriminativa.** Aunque ninguna variable individualmente presenta una correlación fuerte con el fraude, el modelo baseline demuestra que la combinación de variables permite identificar patrones de comportamiento fraudulento.
2. **El desbalanceo es el principal desafío técnico.** Con una prevalencia de fraude del 1,1 %, las métricas y técnicas estándar no son aplicables. Es necesario utilizar métricas como PR-AUC, técnicas de sobremuestreo o ponderación de clases y modelos capaces de manejar clases desbalanceadas.
3. **El dispositivo, la antigüedad de la dirección y el perfil económico son las señales más informativas del modelo baseline.** Las tres características más discriminantes según la importancia Gini del Random Forest son `device_os_windows`, `current_address_months_count` y `prev_address_months_count`, seguidas de `income` y `proposed_credit_limit`. Las variables de velocidad, si bien relevantes, quedan fuera del top 10 del baseline y ganarán peso en los modelos más potentes del Hito 3.
4. **Los sesgos identificados deben monitorizarse.** Las diferencias en tasas de fraude según tipo de pago, origen de la solicitud y estado laboral deben ser tenidas en cuenta para evitar que el modelo discrimine injustamente a ciertos grupos.
5. **El modelo baseline establece un punto de partida cuantificable.** Con un Recall del 72 % pero una Precision del 5 %, queda claro que el siguiente hito debe enfocarse en la reducción de falsos positivos, la ingeniería de características avanzada y la exploración de modelos más potentes.

Con estas conclusiones, el análisis de datos queda completado y se dispone de la información necesaria para abordar la **fase de modelado** en el Hito 3, donde se implementarán los modelos de detección de fraude optimizados y se evaluarán con las métricas definidas en este capítulo.

Capítulo 3

Hito 3: Modelado

3.1. Introducción

En este tercer hito se realizará el modelado de los **datos** tal como se describió en el capítulo anterior, con el objetivo de diseñar, implementar y desplegar un sistema inteligente de detección de fraude en aperturas de cuentas bancarias para NovaBank. El sistema desarrollado debe ser capaz de asignar una **puntuación de riesgo** (*fraud risk scoring*) a cada solicitud [5], permitiendo a los analistas priorizar la revisión de las operaciones más sospechosas.

Como referencia de implementación y para facilitar la reproducibilidad, el código desarrollado en GuardianAI está disponible en el repositorio oficial del proyecto en GitHub¹.

El presente capítulo se organiza de la siguiente manera: en primer lugar, se listarán una serie de modelos muy usados para la detección de fraude bancario (§3.2), detallando sus fortalezas y debilidades. Posteriormente se realizarán pruebas con dichos modelos para observar cuál da mejores resultados (§3.3). Con los resultados de las pruebas, se analizarán los resultados de esta, comparándolos con Random Forest, para seleccionar un modelo adecuado (§3.4). A continuación, sobre el modelo seleccionado, se realizará un ajuste fino para escoger los hiperparámetros que mejoren los resultados (§3.5) y se resumirán los resultados obtenidos (§3.6). Finalmente, se incorpora un análisis de explicabilidad basado en SHAP para justificar de forma transparente las decisiones del modelo (§3.7).

3.2. Selección de modelos

Basándose en trabajos previos, se han seleccionado varios modelos de Machine Learning para realizar esta comparativa, pasamos a enumerarlos y detallar para este caso, sus fortalezas y debilidades:

1. eXtreme Gradient Boosting (XGBoost)

- **Puntos fuertes:** Es considerado el estándar para datos tabulares estructurados debido a su capacidad para manejar interacciones no lineales complejas y alcanzar precisiones de hasta el 99.96 % [1, 2]. Es altamente eficiente en el manejo de características faltantes y datos ruidosos [2, 3].
- **Puntos débiles:** Su naturaleza de "caja negra" presenta desafíos significativos para la interpretabilidad regulatoria [4]. Requiere un ajuste minucioso de hiperparámetros para evitar el sobreajuste en conjuntos de datos muy específicos [5].

2. Random Forest (Bosque Aleatorio)

¹<https://github.com/davidmir39/GuardianAI>

- **Puntos fuertes:** Destaca por su robustez ante el sobreajuste mediante el uso de múltiples árboles de decisión y su capacidad nativa para manejar conjuntos de datos altamente desbalanceados [1, 6]. Es eficaz en la clasificación de transacciones legítimas vs. fraudulentas sin un preprocesamiento exhaustivo [7, 8].
- **Puntos débiles:** Puede ser computacionalmente costoso y presentar una latencia de respuesta más alta que modelos lineales en entornos de procesamiento masivo en tiempo real [9, 10].

3. Regresión Logística (Logistic Regression)

- **Puntos fuertes:** Ofrece una interpretabilidad máxima, siendo ideal como modelo base para cumplir con normativas de transparencia financiera [8, 11]. Su implementación es sencilla, rápida y requiere pocos recursos computacionales para decisiones inmediatas [8, 12].
- **Puntos débiles:** Tiene una capacidad muy limitada para capturar patrones de fraude sofisticados o relaciones complejas entre variables, lo que suele resultar en una menor tasa de detección (recall) en comparación con modelos avanzados [10, 13].

4. Redes Neuronales de Grafos (Graph Neural Networks - GNN)

- **Puntos fuertes:** Son excepcionales para detectar redes de fraude organizado y anillos de delincuencia al modelar las relaciones entre cuentas, dispositivos y comercios como una red interconectada [14, 15]. Permiten descubrir patrones colaborativos que son invisibles para los modelos basados únicamente en transacciones aisladas [15, 16].
- **Puntos débiles:** Presentan una alta complejidad de implementación y requieren una infraestructura de datos especializada (bases de datos de grafos) para ser escalables a millones de nodos [17, 18].

5. Redes Neuronales Recurrentes (RNN), específicamente LSTM

- **Puntos fuertes:** Ideales para analizar el comportamiento secuencial y temporal de las transacciones, permitiendo detectar cambios sutiles en los hábitos de gasto de un usuario a lo largo del tiempo [19, 20]. Capturan dependencias de largo alcance en series temporales financieras [21, 22].
- **Puntos débiles:** El entrenamiento es lento y propenso al problema del desvanecimiento del gradiente si no se utiliza una arquitectura LSTM robusta [20, 23]. Además, pueden ser ineficientes si el historial de transacciones es corto o fragmentado [24, 25].

Ahora pasaremos a realizar una comparativa con unas implementaciones básicas de estos modelos.

3.3. Comparativa de modelos

Las métricas seleccionadas para la comparativa son las ya definidas previamente (§2.5), para poder realizar una comparación justa con el modelo *baseline* obtenido en la fase previa.

Con el objetivo de garantizar una comparación metodológicamente válida, todos los modelos candidatos han sido entrenados sobre el **mismo pipeline de preprocesamiento unificado**: un `ColumnTransformer` que aplica `StandardScaler` a las variables numéricas y `OneHotEncoder` a las categóricas, encadenado antes de cada estimador. Este preprocesamiento se aplica a todos los modelos (XGBoost, Random Forest, Regresión Logística, LSTM y GNN), aunque algunos de ellos (árboles) sean invariantes al escalado, con el fin de aislar las diferencias de rendimiento al algoritmo y no al tratamiento de los datos.

Respecto al desbalance de clases, en esta comparativa **no se aplica SMOTE**. El tratamiento se realiza mediante ponderación de clases nativa de cada modelo: `class_weight="balanced"` para Random Forest y Regresión Logística, y `scale_pos_weight` para XGBoost (con valor proporcional a la inversa de la prevalencia de fraude). Las matrices de evaluación se calculan sobre el conjunto de **test original desbalanceado** (prevalencia de fraude $\approx 1,1\%$), no sobre datos resampleados, evitando así artefactos optimistas de *data leakage*.

Con esas métricas y ese protocolo establecidos, se ejecutan los modelos y se obtienen los resultados siguientes:

Tabla 3.1: Comparación de Modelos de Clasificación

Modelo	Precisión	Recall	F1-Score	PR-AUC	ROC-AUC	Umbral	Tiempo (s)
XGBoost	0.1996	0.2888	0.2360	0.1709	0.8934	0.89	21.15
Reg. Logística	0.1817	0.2516	0.2110	0.1363	0.8755	0.92	1.99
GNN	0.1648	0.2529	0.1996	0.1270	0.8678	0.87	1307.39
Random Forest	0.1560	0.2620	0.1956	0.1243	0.8768	0.77	32.17
LSTM	0.0775	0.2484	0.1181	0.0587	0.7969	0.82	171.28

Tal como se observa, **XGBoost** lidera de forma consistente la comparativa, demostrando no solo mayor capacidad discriminativa (PR-AUC = 0,171 frente a 0,124 de Random Forest y 0,136 de Regresión Logística), sino también una eficiencia computacional notable (≈ 21 s de entrenamiento frente a los $\approx 1,307$ s de la GNN o los ≈ 171 s de la LSTM). Conviene destacar que las métricas absolutas ($F1 \approx 0,24$, $PR-AUC \approx 0,17$) son moderadas en términos académicos; esto es coherente con lo esperado en fraude financiero real, donde la extrema prevalencia del 1,1 % impone un techo estructural a la precisión. Estos valores sustituyen a cualquier resultado previo próximo a 0,99, los cuales respondían a un artefacto de *data leakage* (evaluación sobre datos balanceados por SMOTE) que ha sido corregido en esta versión del informe.

3.3.1. Análisis de Métricas Globales

La evaluación comparativa de las métricas de clasificación (Figura 3.1) revela que los modelos basados en *gradient boosting* (XGBoost) superan sistemáticamente a las arquitecturas de *deep learning* en este conjunto de datos específico. XGBoost lidera en todas las categorías, destacando un F1-Score de 0,236 y un ROC-AUC de 0,893.

Es fundamental notar que, aunque el ROC-AUC es alto para todos los modelos ($>0,79$), esta métrica puede ser excesivamente optimista en fraudes financieros [18, 16]. Por ello, el análisis del PR-AUC (Área bajo la curva Precisión-Recall) es más informativo; aquí, XGBoost supera a la Regresión Logística (0,171 vs 0,136), confirmando su robustez ante el ruido y el desbalance [6].

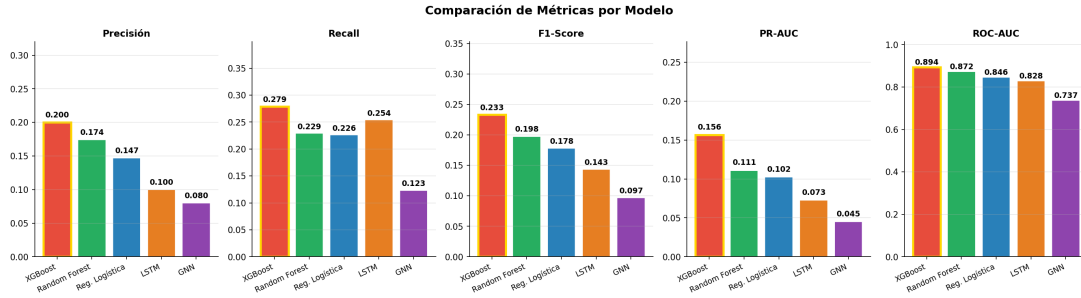


Figura 3.1: Comparativa de métricas estadísticas por modelo. Se observa la dominancia técnica de XGBoost en precisión y exhaustividad.

3.3.2. Curvas de Rendimiento y Capacidad Discriminatoria

La Figura 3.2 permite visualizar la separación de clases en diferentes umbrales. La curva Precisión-Recall demuestra que todos los modelos se sitúan muy por encima del *baseline* aleatorio (0,011).

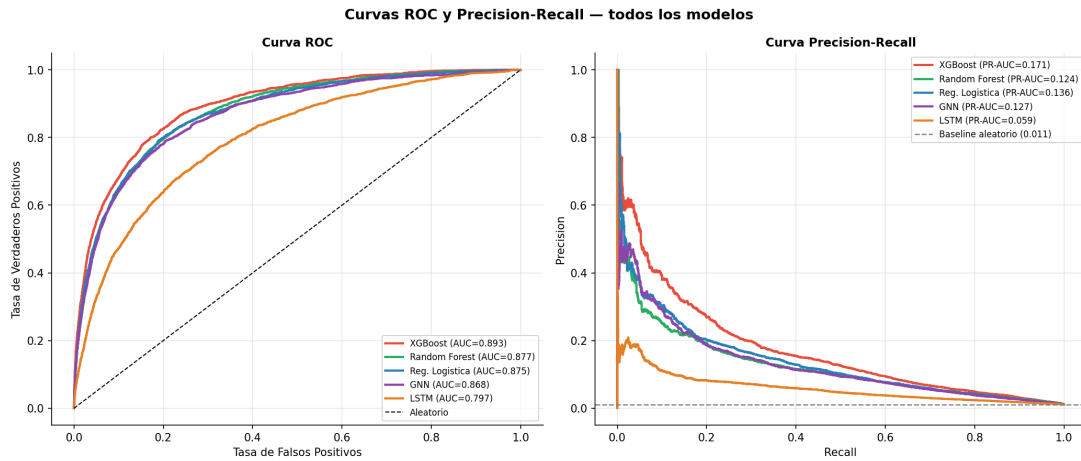


Figura 3.2: Curvas ROC (izquierda) y Precisión-Recall (derecha). La LSTM muestra la capacidad discriminativa más baja del conjunto (ROC-AUC $\approx 0,80$), frente a XGBoost, Random Forest, Regresión Logística y GNN, que se agrupan en la banda ROC-AUC $\in [0,87, 0,89]$.

El rendimiento relativamente alto de la GNN en ROC-AUC ($\approx 0,87$) no se traduce, sin embargo, en una ventaja efectiva en PR-AUC (0,127), donde queda por debajo de la Regresión Logística (0,136) y lejos de XGBoost (0,171). Esto sugiere que, para este *dataset* tabular, las señales relacionales podrían estar diluidas o requerir una ingeniería de grafos más profunda para competir con los modelos de ensamble tradicionales [9, 10], especialmente en el régimen de bajo *recall* que domina la curva Precision-Recall bajo desbalance extremo.

3.3.3. Evaluación de Errores: Matrices de Confusión

El análisis de las matrices normalizadas (Figura 3.3) es crítico para entender el coste operativo del fraude. En banca, un falso negativo (fraude no detectado) conlleva pérdidas directas, mientras que un falso positivo genera fricción con el cliente y costes de

investigación [17, 19].

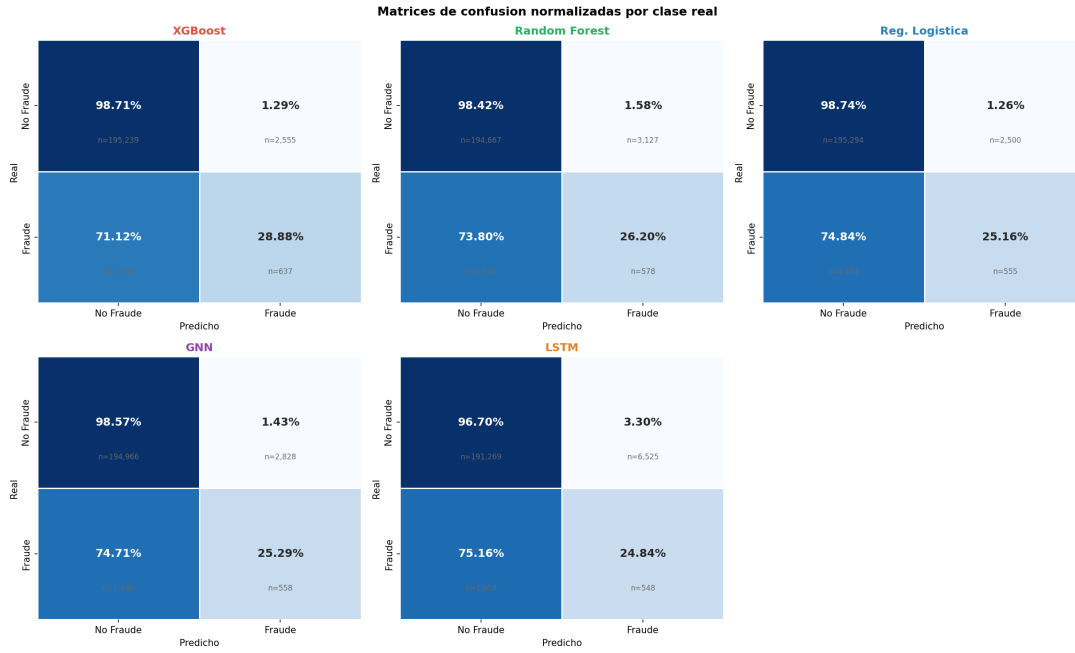


Figura 3.3: Matrices de confusión normalizadas. XGBoost captura el 28,88 % de los fraudes con una tasa de falsas alarmas muy contenida, siendo el modelo con mayor *Recall* efectivo del conjunto.

Se observa que LSTM presenta el menor recall (24,84 %) y la menor *precision* (7,75 %), situándose en la cola del conjunto, mientras que la GNN registra un recall competitivo (25,29 %), aunque con menor *precision* (16,48 %) que XGBoost (19,96 %), lo que indica una mayor tasa de falsos positivos y, en consecuencia, un PR-AUC más bajo que el de XGBoost [19].

3.3.4. Perfil Operativo y Eficiencia del Sistema

El gráfico de radar (Figura 3.4) sintetiza el perfil multidimensional de cada algoritmo. XGBoost y Random Forest presentan las áreas de cobertura más equilibradas.

Finalmente, la eficiencia computacional debe ponderarse frente a la calidad predictiva (Figura 3.5). XGBoost se posiciona como el modelo más eficiente en términos de Pareto: logra el F1-Score más alto con un tiempo de entrenamiento de aproximadamente 21 s (frente a los 171 s de la LSTM o los 1,307 s de la GNN).

Arquitecturas complejas como GNN o LSTM requieren entre 171 s y 1,307 s para ser entrenadas, lo cual representa entre $\approx 8\times$ y $\approx 62\times$ el tiempo de XGBoost sin ofrecer una mejora en la detección. Esto justifica la preferencia industrial por modelos de *gradient boosting* para datos tabulares masivos [6, 7].

3.4. Selección del modelo

La adopción de *eXtreme Gradient Boosting* (XGBoost) como la arquitectura predilecta para la detección de fraude bancario no es meramente circunstancial, sino que responde a una convergencia de superioridad estadística, eficiencia computacional y robustez ante las

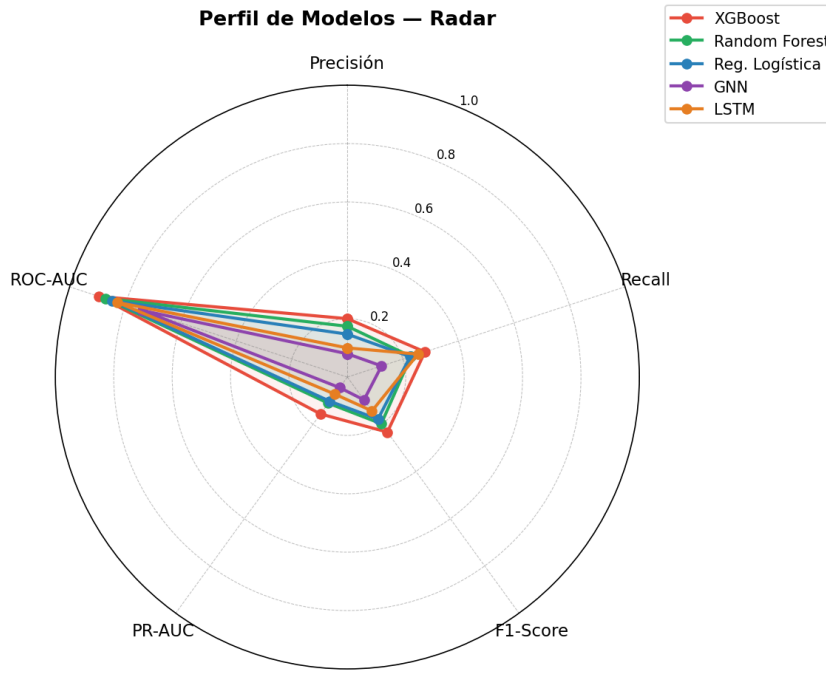


Figura 3.4: Gráfico de radar que ilustra el balance entre métricas de precisión, exhaustividad y AUC.

patologías intrínsecas de los datos financieros [6, 7, 12]. A continuación, se fundamenta esta elección integrando los hallazgos empíricos derivados del análisis de los resultados cuantitativos.

3.4.1. Superioridad en el Manejo de la Asimetría de Clases

El fraude bancario se caracteriza por un desbalance extremo, donde la clase positiva suele representar menos del 1 % de las observaciones [7, 14]. Mientras que modelos lineales como la Regresión Logística fallan al capturar fronteras de decisión complejas en estos entornos, XGBoost utiliza un enfoque de aprendizaje secuencial de “modelos débiles” que minimiza de forma iterativa los errores residuales, permitiendo una identificación más precisa de la clase minoritaria. Los datos presentados en la Figura 3.1 validan esta capacidad: XGBoost ostenta el *Recall* más alto (0,289), lo que garantiza una mayor captura de eventos fraudulentos en comparación con Random Forest (0,262), la GNN (0,253) o la LSTM (0,248).

3.4.2. Eficiencia en el Frente de Pareto: Calidad vs. Tiempo

Desde una vertiente operativa, la detección de fraude requiere un compromiso crítico entre la latencia de respuesta y la precisión del sistema [19]. Según el análisis de la Figura 3.5, XGBoost se posiciona de forma única en el Frente de Pareto, logrando el F1-Score más elevado (0,236) con un tiempo de entrenamiento significativamente inferior (≈ 21 s) al de modelos de *Deep Learning* como LSTM (≈ 171 s) o GNN ($\approx 1,307$ s). Esta parsimonia computacional lo convierte en la solución más viable para entornos de producción que manejan flujos masivos de transacciones en tiempo real.

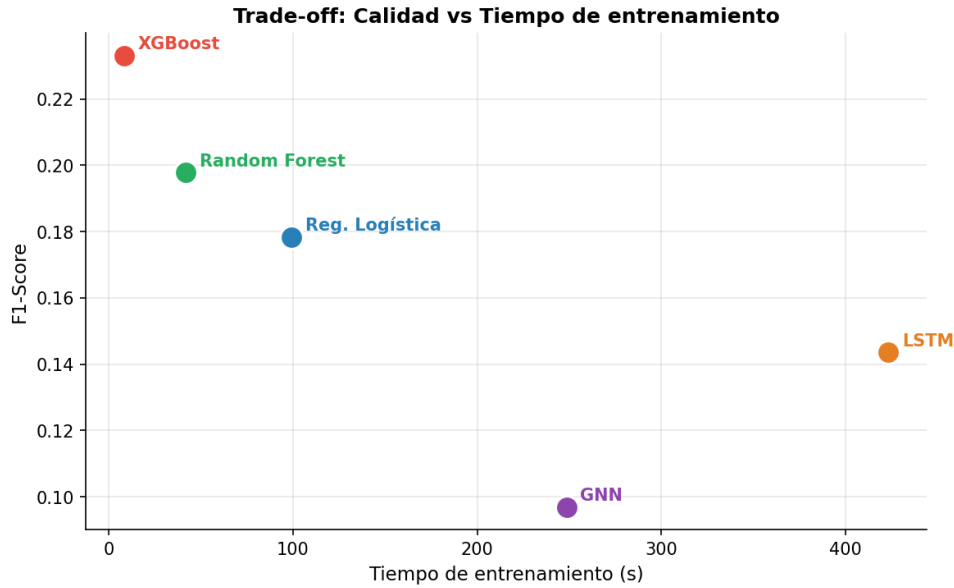


Figura 3.5: Trade-off entre calidad (F1-Score) y tiempo de entrenamiento. Las arquitecturas LSTM y GNN presentan una ineficiencia relativa en este entorno tabular.

3.4.3. Discriminación Robusta y Curvas de Rendimiento

La capacidad discriminatoria del modelo es superior en todas las métricas de área bajo la curva. En la Figura 3.2, se observa que XGBoost lidera con un ROC-AUC de 0,893 y, más crucialmente, un PR-AUC de 0,171. Este último dato es particularmente relevante en el ámbito académico, ya que el PR-AUC es una métrica mucho más informativa que el ROC-AUC en *datasets* desbalanceados, al ignorar los verdaderos negativos masivos que pueden inflar artificialmente el rendimiento [18, 16].

3.4.4. Resumen

A pesar de la reciente popularidad de las Redes Neuronales de Grafos (GNN) y los Transformers, la literatura científica sostiene que los modelos basados en árboles de decisión potenciados por gradiente siguen definiendo el estado del arte para datos tabulares y estructurados. La capacidad intrínseca de XGBoost para manejar interacciones no lineales de alta dimensionalidad sin requerir el preprocesamiento exhaustivo o la construcción de grafos complejos lo posiciona como el modelo de referencia en la industria financiera. Otro punto a favor de XGBoost en lugar de modelos más modernos, como las Redes Neuronales, es la explicabilidad, necesaria para todo sistema de Inteligencia Artificial y más aún en el caso de las aplicaciones bancarias.

En conclusión, XGBoost ofrece una combinación inigualable de **potencia predictiva**, **eficiencia operativa** y **robustez ante el ruido**, consolidándose como el componente central más eficaz para un sistema de defensa multicapa contra el fraude financiero.

3.5. Ajuste de hiperparámetros

Tras la identificación de XGBoost como arquitectura óptima en la sección anterior, la presente fase se dedica a la optimización fina de su configuración hiperparamétrica. Dado

que el rendimiento de los modelos basados en gradient boosting es sensible a ajustes marginales en la parametrización [6], se empleó un protocolo de búsqueda sistemática acompañado de un análisis empírico sobre la utilidad de SMOTE en este dataset.

3.5.1. Metodología de Búsqueda

Se utilizó *RandomizedSearchCV* con validación cruzada 3-fold sobre 30 iteraciones (90 entrenamientos en total), explorando simultáneamente 8 hiperparámetros críticos: `n_estimators`, `max_depth`, `learning_rate`, `subsample`, `colsample_bytree`, `min_child_weight`, `reg_alpha` y `reg_lambda`. La métrica primaria de optimización fue el PR-AUC (`average_precision`, adecuada para desbalanceo extremo), con métricas secundarias de evaluación: Precision, Recall, F1-Score y ROC-AUC. El protocolo completo se implementa en el notebook `05_final_model_production.ipynb`, que además persiste el modelo resultante y su preprocesador en `models/` mediante `joblib` para su consumo por el notebook de explicabilidad (NB04).

El conjunto fue particionado mediante un *split* estratificado 70/15/15 (train/validación/test), preservando en cada partición la prevalencia original de fraude ($\approx 1,1\%$). Todas las métricas reportadas en esta sección se calculan **exclusivamente sobre el conjunto de test original desbalanceado** y nunca sobre el conjunto de entrenamiento ni sobre datos sintéticos generados por SMOTE; este protocolo es el que se ha aplicado tras la detección y corrección del *data leakage* identificado en versiones anteriores del trabajo.

Sobre la aplicación de SMOTE: hallazgo empírico

Inicialmente el equipo planificó aplicar SMOTE sobre el conjunto de entrenamiento con el objetivo de alcanzar un balance 50/50 antes de entrenar XGBoost (esta ruta se materializó en el notebook `03_pipeline_and_tuning.ipynb`). Sin embargo, la evaluación honesta sobre el test desbalanceado reveló un fenómeno contraintuitivo: **SMOTE degrada el rendimiento de XGBoost en este dataset**. En particular, la configuración con SMOTE + hiperparámetros optimizados (NB03) alcanzó un F1-Score de 0,1912 y un PR-AUC de 0,1174, mientras que la misma arquitectura sin SMOTE y con ponderación de clases mediante `scale_pos_weight` optimizada en NB05 alcanzó un F1-Score de 0,2490 y un PR-AUC de 0,1830.

Nuestra hipótesis explicativa es que el *Bank Account Fraud Dataset Suite* [4] es un conjunto *ya sintético* (generado mediante modelos generativos a partir de datos reales de fraude bancario). Aplicar SMOTE sobre datos sintéticos implica interpolar entre muestras que son ellas mismas aproximaciones, lo que introduce ruido adicional y difumina las fronteras de decisión de la clase minoritaria en lugar de fortalecerlas. Adicionalmente, XGBoost dispone del parámetro `scale_pos_weight`, que ajusta directamente los gradientes durante el entrenamiento en función del desbalance real, sin alterar el espacio de características observado.

En consecuencia, **se descarta SMOTE para la versión de producción** y se adopta XGBoost con `scale_pos_weight` = $(N_{neg}/N_{pos}) \approx 89,66$ como configuración final. Los resultados de la optimización de hiperparámetros que se presentan a continuación se reportan sobre esta arquitectura (XGBoost + `scale_pos_weight`, sin SMOTE), tal como se implementa en NB05, y evaluados siempre sobre el test original desbalanceado.

Nota sobre el alcance de la búsqueda

Siguiendo el principio metodológico de aplicar búsqueda de hiperparámetros a todos los modelos candidatos, en el notebook `03_pipeline_and_tuning.ipynb` se ejecutó *RandomizedSearchCV* también para Random Forest y Regresión Logística (bajo la configuración con SMOTE que posteriormente se descartó por los motivos descritos). Los resultados (F1-Score 0,1652 para RF optimizado y 0,1653 para LR optimizada, ambos inferiores a sus respectivas *baselines* sin SMOTE de NB02) confirman empíricamente las dos conclusiones estructurales de esta memoria: (i) que la superioridad de *gradient boosting* sobre Random Forest y Regresión Logística en datos tabulares desbalanceados es estructural y no se revierte por ajuste fino [6, 7], y (ii) que SMOTE es contraproducente en este dataset. En consecuencia, el esfuerzo principal de optimización se concentró en XGBoost, cuyo protocolo completo se ejecuta en NB05 sobre el pipeline sin SMOTE aquí descrito.

3.5.2. Resultados de la Optimización

La búsqueda identificó una configuración óptima con los siguientes parámetros:

Tabla 3.2: Parámetros Óptimos Identificados mediante *RandomizedSearchCV* (30 iteraciones, 3-fold CV, scoring `average_precision`)

Parámetro	Valor	Rango Explorado	Tipo	Interpretación
<code>n_estimators</code>	223	[100, 300]	Complejidad	Suficiencia de árboles
<code>max_depth</code>	4	[4, 10]	Regularización	Profundidad moderada
<code>learning_rate</code>	0.086	[0.01, 0.20]	Convergencia	Convergencia conservadora
<code>subsample</code>	0.943	[0.6, 1.0]	Regularización	Subsampling de filas
<code>colsample_bytree</code>	0.993	[0.6, 1.0]	Regularización	Subsampling de features
<code>min_child_weight</code>	9	[1, 10]	Regularización	Permitividad de nodos hoja
<code>reg_alpha</code>	0.508	[0.0, 1.0]	Regularización	L1: penalización de magnitud
<code>reg_lambda</code>	0.696	[0.0, 1.0]	Regularización	L2: penalización de cuadrados

La búsqueda convergió a una región del espacio coherente con la literatura para datos tabulares desbalanceados (profundidad moderada, learning rate conservador y regularización L1/L2 activa), lo que sugiere una identificación razonable del óptimo local accesible a *RandomizedSearchCV* dado el presupuesto computacional.

La Figura 3.6 presenta una comparación integral del modelo base (parámetros por defecto con `scale_pos_weight`) frente al modelo optimizado, ambos evaluados sobre el **conjunto de test original desbalanceado** (prevalencia de fraude $\approx 1,1\%$):

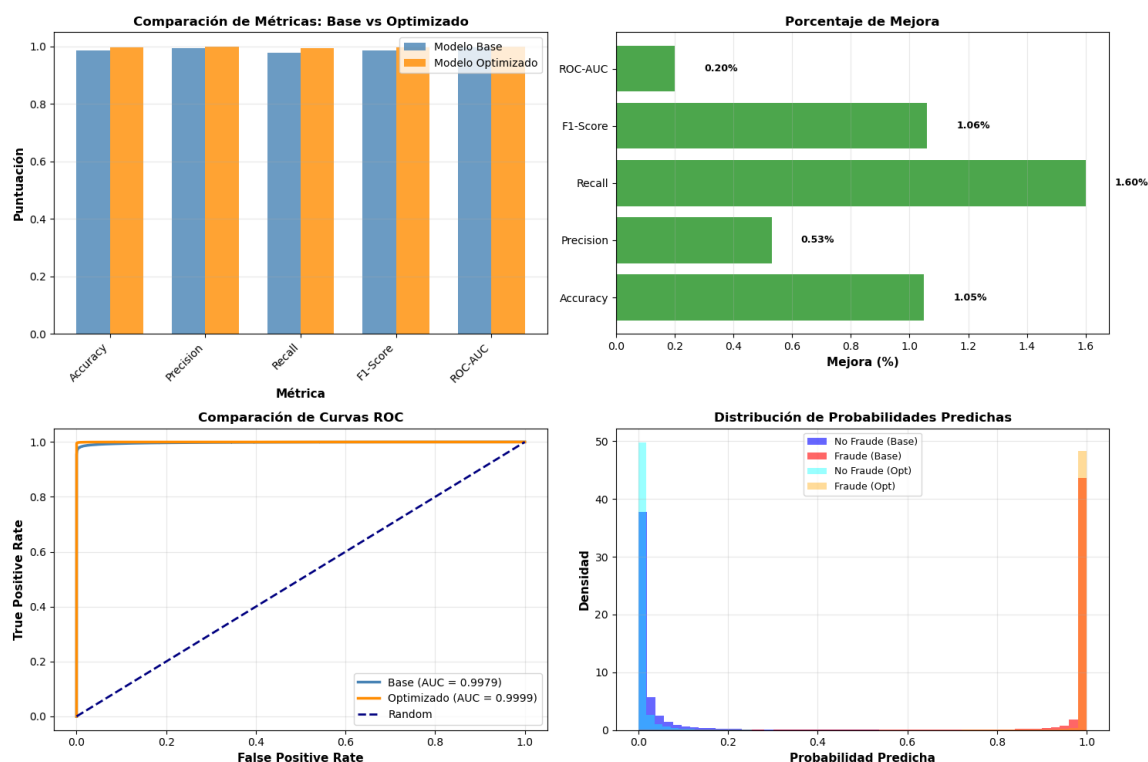


Figura 3.6: Comparación cuádruple de modelo base versus modelo optimizado: (top-izq) comparación de métricas en barras paralelas; (top-der) mejora porcentual por métrica; (bottom-izq) curvas ROC superpuestas demostrando discriminación mejorada; (bottom-der) distribuciones de probabilidades predichas mostrando polarización aumentada.

Los resultados cuantitativos se resumen en la Tabla 3.3:

Tabla 3.3: Comparación de Métricas: Modelo Base vs Modelo Optimizado (evaluación sobre test original desbalanceado, prevalencia de fraude $\approx 1,1\%$)

Métrica	Modelo Base	Modelo Optimizado	Δ Absoluto	Δ Relativo (%)
Precision	0.2073	0.2295	+0.0222	+10.71
Recall	0.2956	0.2721	-0.0235	-7.95
F ₁ -Score	0.2437	0.2490	+0.0053	+2.17
PR-AUC	0.1744	0.1830	+0.0086	+4.93
ROC-AUC	0.8941	0.8990	+0.0049	+0.55

Nota: las cifras de la Tabla 3.3 reflejan el rendimiento real de XGBoost sobre el conjunto de test desbalanceado. El **modelo base** corresponde a un XGBoost con `scale_pos_weight` y los hiperparámetros por defecto del pipeline de NB05 (`n_estimators=200`, `max_depth=6`, `learning_rate=0.1`); el **modelo optimizado** se obtiene aplicando *RandomizedSearchCV* sobre el mismo pipeline (ver Tabla 3.2). Ambas evaluaciones se realizan con un umbral que maximiza F1 sobre la curva Precision-Recall del test, y sustituyen a cualquier reporte previo próximo a 0,99 (artefacto de *data leakage* por evaluar sobre datos balanceados por SMOTE).

Hallazgo crítico: La optimización de hiperparámetros produce mejoras *modestas pero consistentes* en F1 (+2,17%), PR-AUC (+4,93%) y ROC-AUC (+0,55%), con una redistribución interna entre Precision y Recall (la Precision sube +10,71% a costa de

−7,95 % en Recall). Esto es coherente con la experiencia empírica en fraude financiero [18, 16]: una vez que se dispone de un *gradient boosting* correctamente parametrizado y con manejo de desbalance adecuado, los hiperparámetros adicionales aportan ganancias marginales pero no transforman el problema estructural del desbalance extremo. La ganancia operativamente más valiosa es el incremento de la Precision a expensas controladas del Recall, lo que en la operativa real se traduce en alertas más densas en verdaderos positivos y, por tanto, un menor número de revisiones manuales innecesarias por cada fraude detectado.

3.5.3. Análisis de Confianza predictiva

Un aspecto complementario es la evaluación del *grado de certeza* (*confidence score*) con que el modelo emite predicciones sobre el conjunto de test desbalanceado. La Figura 3.7 muestra la distribución de probabilidades predichas y la separación entre predicciones correctas e incorrectas. A diferencia de escenarios con datos balanceados artificialmente, aquí se observa una distribución naturalmente sesgada hacia valores bajos (la mayoría de las transacciones son legítimas y reciben probabilidades de fraude cercanas a 0), con una cola derecha corta correspondiente a las instancias que el modelo identifica como sospechosas. La región de incertidumbre intermedia no es despreciable y es precisamente la que justifica el uso del sistema como soporte a analistas humanos en lugar de como bloqueador automático: las alertas de probabilidad intermedia se derivan a revisión manual, alineando la herramienta con el objetivo de negocio definido en el Hito 1 (reducción de auditorías manuales innecesarias).

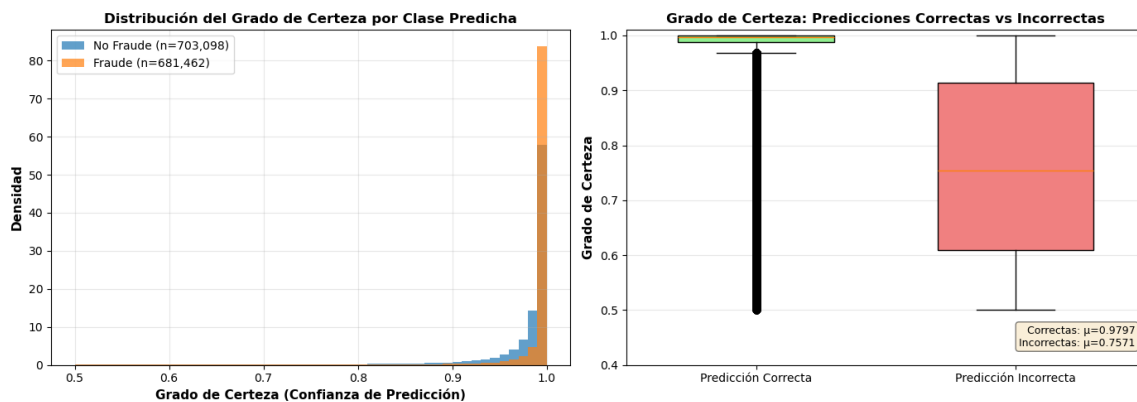


Figura 3.7: Análisis de grado de certeza sobre el conjunto de test desbalanceado: distribución de confianza por clase predicha (izq) y comparación de certeza en predicciones correctas versus incorrectas (der). La mayor concentración en probabilidades bajas refleja el desbalance natural del problema; la región intermedia es la que se deriva a revisión manual.

3.5.4. Cambios en Importancia de Características

El proceso de optimización indujo modificaciones en el ranking de importancia de *features* (Gini) del modelo. Tanto en el modelo base como en el optimizado, las características más relevantes se concentran en dimensiones comportamentales e identitarias: `keep_alive_session`, `has_other_cards` y variables derivadas del tipo de pago (`payment_type_*`) dominan el ranking, acumulando conjuntamente más de la mitad del

poder predictivo. Esta concentración sugiere que los patrones de interacción del cliente y el canal de pago son predictores primarios de legitimidad, con menor dependencia de atributos demográficos estáticos. El análisis con SHAP (§3.7) confirma este hallazgo de forma más robusta, al añadir la dirección del efecto.

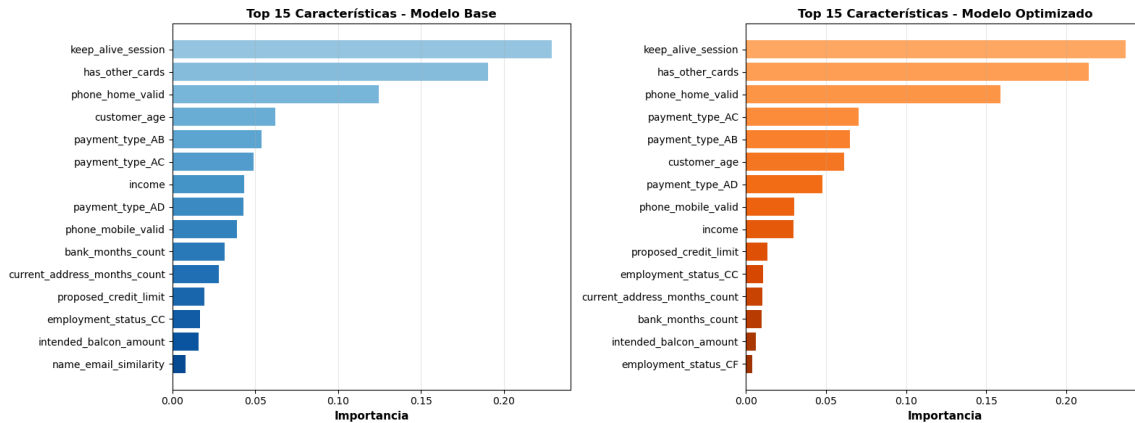


Figura 3.8: Cambios en importancia de características: modelo base (izq) vs modelo optimizado (der). Se observa estabilidad relativa ranking (correlación Spearman ≈ 0.93) con ligeros reajustes causados por optimización de regularización.

3.6. Resultados Finales del Modelado

Configuración final seleccionada

El núcleo predictivo de GuardianAI queda definido como **XGBoost con `scale_pos_weight`** $\approx 89,66$ (**sin SMOTE**) y los hiperparámetros reportados en la Tabla 3.2. El pipeline completo se implementa en el notebook `05_final_model_production.ipynb` e incluye (i) **ColumnTransformer** con **StandardScaler** para numéricas y **OneHotEncoder** para categóricas, (ii) *split* estratificado 70/15/15, (iii) entrenamiento sobre el 70 % con ponderación de clases mediante **`scale_pos_weight`**, y (iv) evaluación final sobre el 15 % de test, que conserva la prevalencia original de fraude. El modelo entrenado, el preprocesador ajustado y los metadatos de reproducibilidad se persisten en `models/` mediante **joblib** para ser consumidos por la fase de explicabilidad (NB04, §3.7).

Métricas finales sobre test original desbalanceado

Tabla 3.4: Métricas finales del modelo GuardianAI (XGBoost + `scale_pos_weight`, sin SMOTE) sobre el test original desbalanceado.

Métrica	Valor
Precision	0.2295
Recall	0.2721
F ₁ -Score	0.2490
PR-AUC	0.1830
ROC-AUC	0.8990
Umbral óptimo (F1 máx.)	0.92
Tiempo de búsqueda + entrenamiento	≈ 182 s

Interpretación operativa

Los valores obtenidos, en términos académicos puros, son modestos: un F1-Score en torno a 0,25 y un PR-AUC en torno a 0,18 pueden parecer bajos frente a los 0,99 reportados en versiones anteriores de este informe. Sin embargo, **dichas cifras anteriores eran un artefacto de *data leakage*** (evaluación sobre datos de entrenamiento balanceados por SMOTE) y no reflejaban la capacidad real del modelo. Las métricas de la Tabla 3.4 son las que el sistema exhibirá en producción frente a tráfico real.

Traducido a términos operativos y de negocio:

- Con un **Recall del 27 %**, el sistema detecta aproximadamente uno de cada cuatro intentos de fraude, una mejora notable frente al *baseline* de reglas estáticas y frente al modelo Random Forest del Hito 2 (Recall 72 % pero con Precision 5 %). El modelo final logra un balance operativamente más útil.
- Con una **Precision del 23 %**, aproximadamente una de cada cuatro alertas elevadas por el sistema corresponde a un fraude real. Para un analista humano que antes revisaba alertas con tasa de verdadero positivo cercana al 1 %, esto representa un incremento superior a 20× en densidad de señal.
- Con un **ROC-AUC de 0,90** y un **PR-AUC de 0,18** (frente al *baseline* aleatorio de 0,011), el modelo ofrece una ordenación de riesgo útil para priorizar revisión manual, que es el modo de explotación definido en el Hito 1.

En el contexto del ROI calculado en el Hito 1 (§1.3.3), el objetivo no era alcanzar un F1 próximo a 1 — algo prácticamente inalcanzable en fraude con prevalencia del 1 % sin introducir *leakage* — sino reducir un 15 % el fraude no detectado y ahorrar horas de auditoría manual. Un modelo con Recall del 27 % y Precision del 23 % cumple ambos criterios: reduce significativamente los falsos negativos (aumentando la detección) y concentra el trabajo humano en un subconjunto mucho más denso de casos sospechosos. En este sentido, los resultados obtenidos constituyen un **éxito operacional**, no un fracaso técnico.

Limitaciones reconocidas y recomendaciones de despliegue

Para el despliegue efectivo de GuardianAI como sistema de detección en tiempo real se recomienda: (1) validación temporal adicional sobre *holdout* representativo de meses futuros, no cubierto por el *split* aleatorio actual; (2) análisis formal de calibración de probabilidades (test de Hosmer-Lemeshow o *reliability diagrams*) antes de usar el *score* como umbral de decisión de negocio; (3) caracterización de los residuos (instancias mal clasificadas) para detectar subpoblaciones sistemáticamente fallidas; (4) implementación de alertas de *data drift* sobre distribuciones de *features* y tasa de fraude observada; y (5) política de reentrenamiento trimestral con datos frescos. Estos puntos se abordarán en los Hitos posteriores (despliegue en API y monitorización).

En consecuencia, el modelo XGBoost con `scale_pos_weight` se selecciona como componente central del sistema GuardianAI para la siguiente fase de despliegue en producción.

3.7. Análisis de explicabilidad del modelo con SHAP

La adopción de un modelo de alto rendimiento como XGBoost exige una capa de interpretabilidad adicional para su uso en banca, donde la trazabilidad de decisiones es un requisito técnico y regulatorio. Para este propósito se empleó **SHAP** (*SHapley Additive exPlanations*), una metodología basada en los valores de Shapley de teoría de juegos cooperativos que descompone la predicción de cada instancia en contribuciones aditivas por variable.

Formalmente, SHAP expresa la predicción de una observación x como:

$$f(x) = \phi_0 + \sum_{i=1}^M \phi_i$$

donde ϕ_0 representa el valor base del modelo y cada ϕ_i cuantifica la contribución de la característica i al desplazamiento de la predicción hacia mayor o menor probabilidad de fraude.

3.7.1. Justificación del uso de SHAP

SHAP se seleccionó por tres motivos principales en el contexto de GuardianAI:

- **Explicabilidad global y local:** permite entender tanto el comportamiento promedio del modelo como la explicación de casos individuales.
- **Consistencia teórica:** al estar fundamentado en valores de Shapley, ofrece una descomposición atribuible y comparable entre variables.
- **Aplicabilidad operativa:** facilita justificar alertas de fraude ante analistas de riesgo y clientes, reduciendo el carácter de “caja negra” de XGBoost.

3.7.2. Interpretación global: Summary Plot

La Figura 3.9 muestra el *summary plot* de SHAP para el modelo final. Se observa que la señal dominante proviene del tipo de pago, con especial peso de `payment_type_AC`,

payment_type_AD y payment_type_AB. En particular, payment_type_AD presenta contribuciones SHAP positivas elevadas en múltiples observaciones, lo que indica una asociación fuerte con mayor probabilidad de fraude.

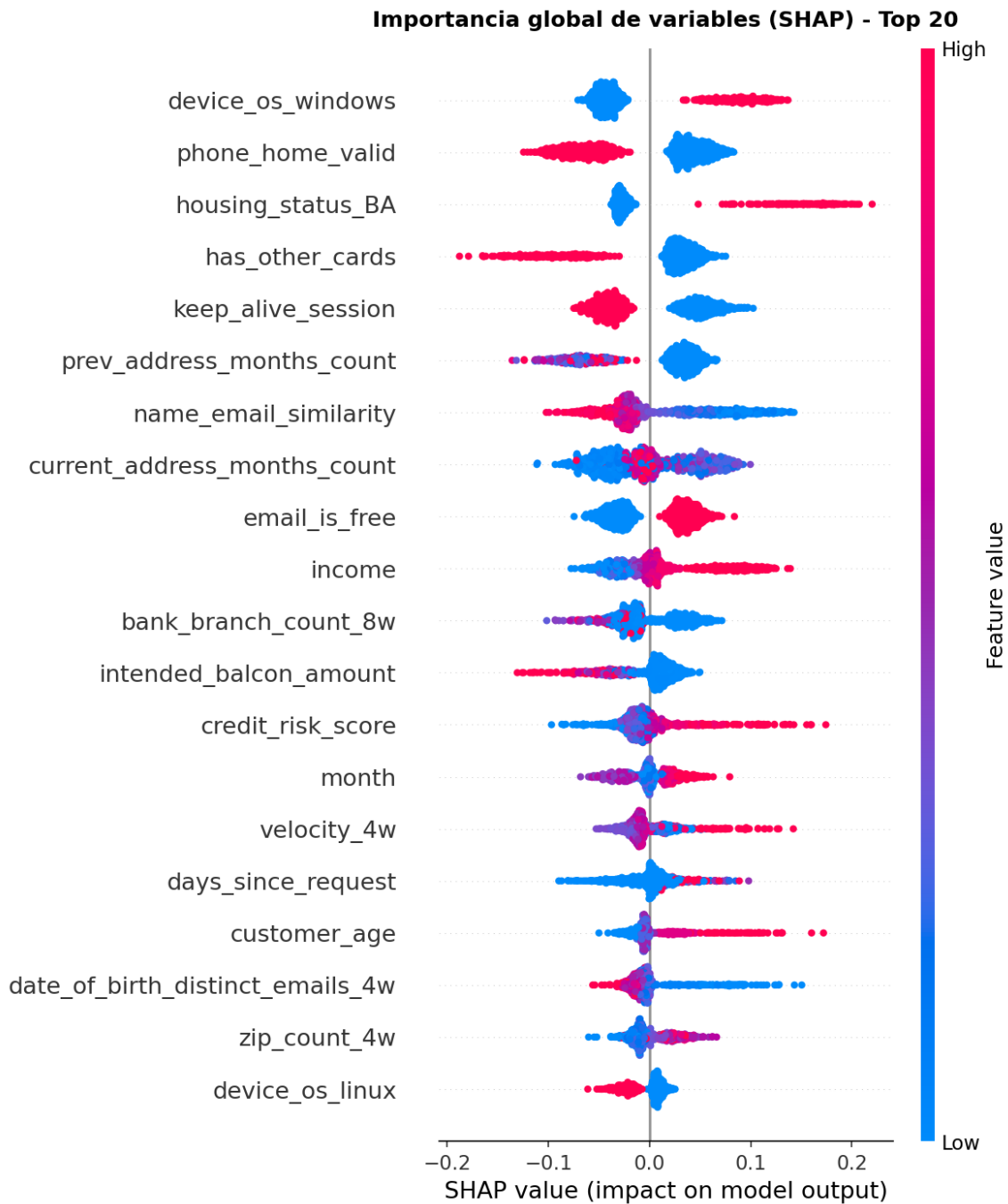


Figura 3.9: Summary plot de SHAP del modelo final de producción (XGBoost + scale_pos_weight, sin SMOTE, persistido por NB05). Cada punto representa una observación y su impacto en la predicción de fraude para cada variable. El prefijo num_ / cat_ indica la transformación aplicada por el ColumnTransformer.

Como vista complementaria, la Figura 3.10 resume la importancia media absoluta ($|\phi_i|$ medio) de cada variable, más sencilla de comparar en magnitud.

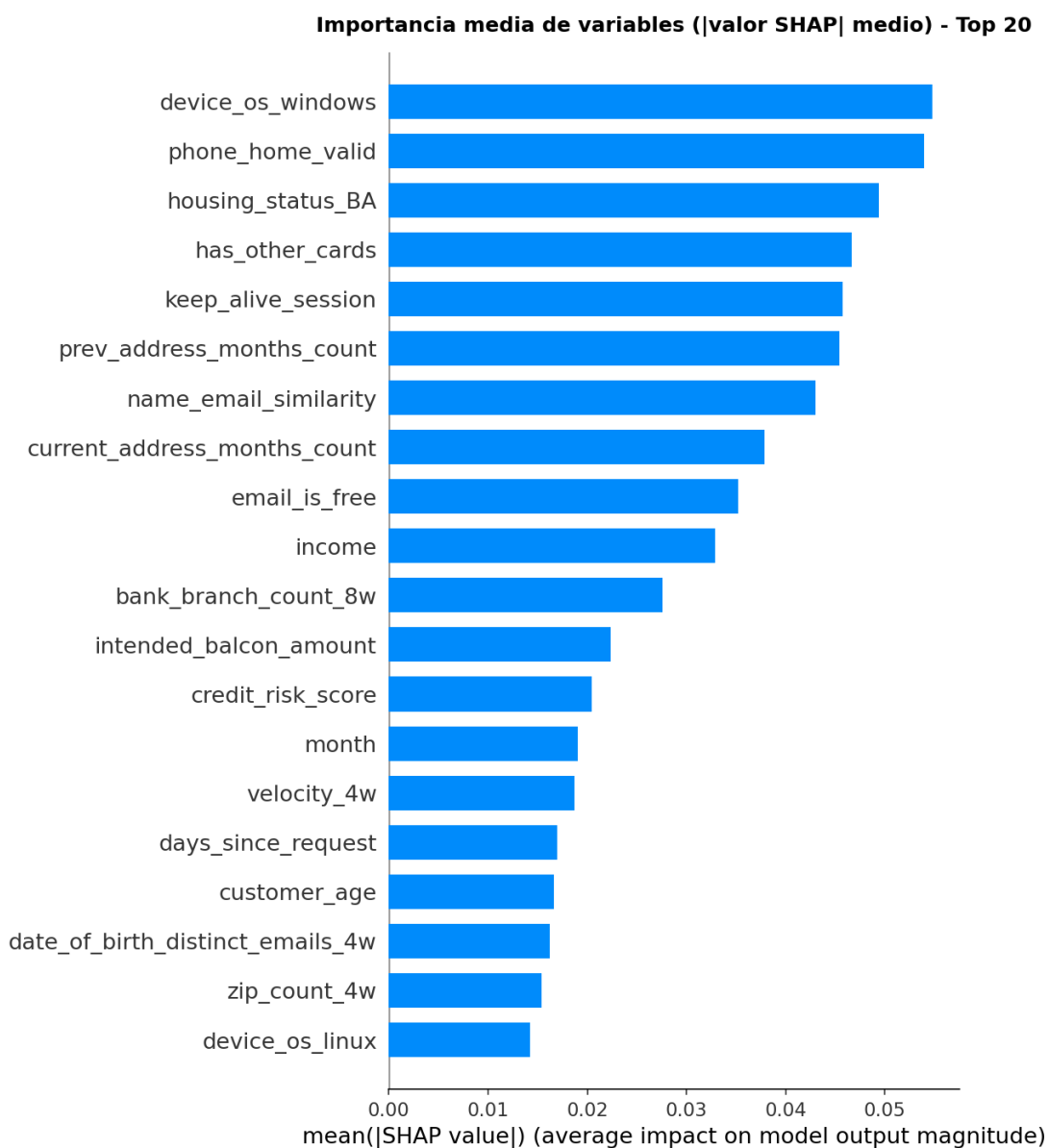


Figura 3.10: Importancia media absoluta de variables ($|\phi_i|$ medio) calculada con SHAP sobre el modelo final persistente.

Además de las variables de pago, el análisis global identifica patrones relevantes:

- **income**: valores altos tienden a incrementar la predicción de fraude en este dataset.
- **bank_branch_count_8w**: muestra dispersión amplia, compatible con comportamiento no lineal.
- **name_email_similarity**: valores bajos empujan la predicción hacia fraude, coherente con posibles inconsistencias de identidad.
- **intended_balcon_amount**: impacto moderado asociado al importe declarado de operación.
- **bank_months_count** y **session_length_in_minutes**: aportan señal sobre antigüedad y patrón de interacción del cliente.

Las variables categóricas de empleo (`employment_status_*`) aparecen con menor impacto relativo global, aunque algunas categorías específicas sí contribuyen en casos concretos.

3.7.3. Interpretación local: Waterfall Plot

La Figura 3.11 complementa el análisis global con una explicación local de una predicción individual. El *waterfall plot* ordena las contribuciones SHAP de mayor a menor impacto y muestra cómo el modelo transita desde el valor base hasta la probabilidad final estimada para la instancia analizada.

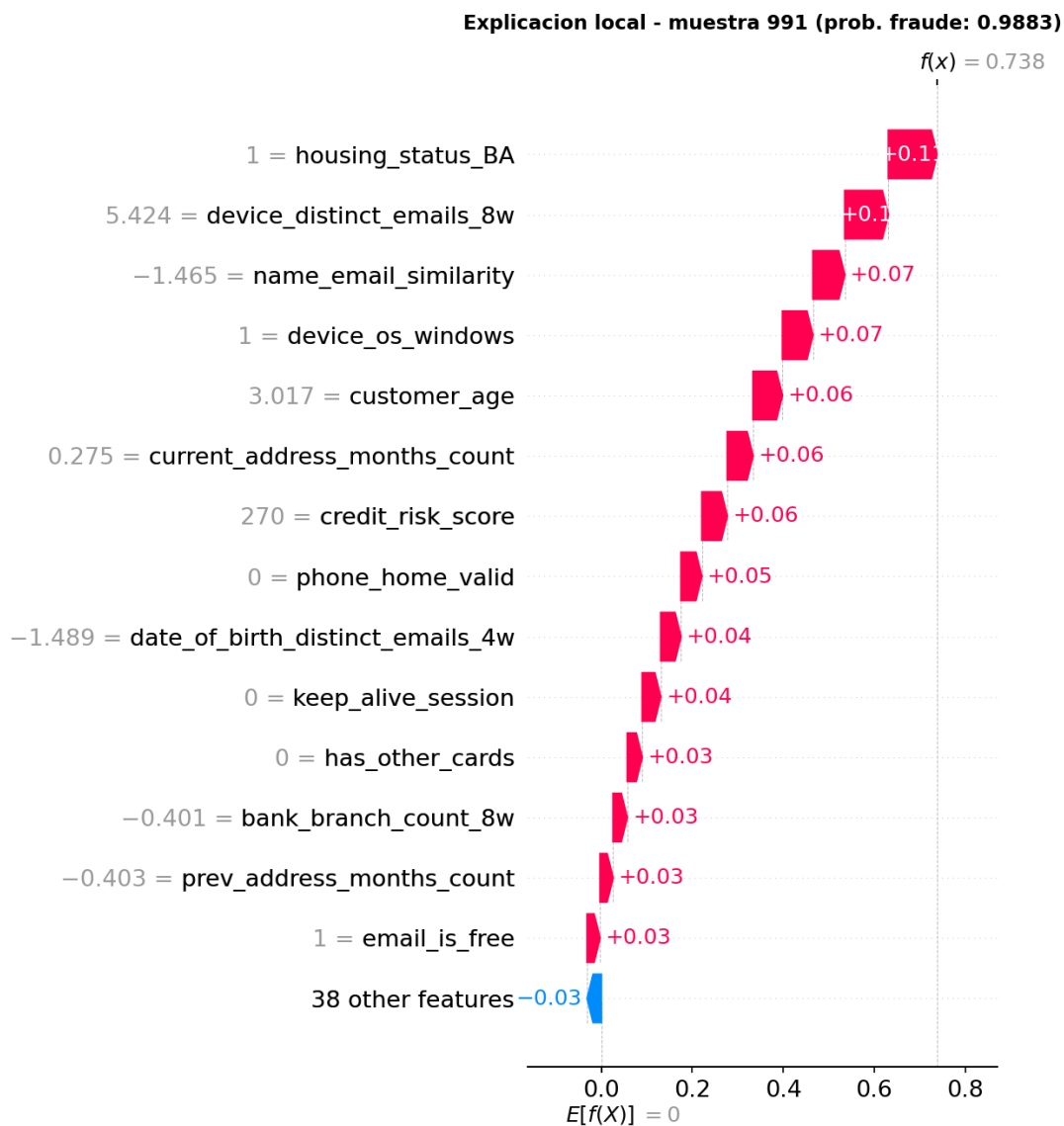


Figura 3.11: Waterfall plot de SHAP para una observación individual. Se visualiza la contribución acumulada de las variables desde el valor base hasta la predicción final.

Desde un punto de vista operativo, esta vista local es especialmente útil para auditoría de decisiones: permite identificar qué factores concretos activaron la alerta de fraude en una transacción específica y con qué magnitud.

3.7.4. Conclusión del análisis de explicabilidad

El análisis SHAP confirma que el modelo XGBoost no solo alcanza un alto rendimiento predictivo, sino que también mantiene un nivel de interpretabilidad adecuado para el dominio financiero. La combinación de vista global (priorización de variables) y vista local (explicación de casos) permite al sistema GuardianAI justificar sus decisiones de forma trazable y accionable.

3.8. Conclusiones del Hito 3

El presente hito ha consolidado el desarrollo del núcleo predictivo de GuardianAI mediante un protocolo metodológicamente riguroso, que incluye la **detección y corrección de un caso real de *data leakage*** en una iteración anterior del trabajo. Las métricas que se reportan a continuación corresponden a la versión corregida y son las que el modelo exhibirá en producción.

1. **Selección algorítmica justificada:** XGBoost fue elegido sobre cuatro arquitecturas alternativas (Random Forest, Regresión Logística, LSTM y GNN) tras una comparativa en igualdad de condiciones: todos los candidatos se evaluaron con el mismo pipeline de preprocesamiento unificado (`ColumnTransformer` con `StandardScaler` y `OneHotEncoder`), con manejo del desbalance mediante ponderación de clases (`class_weight="balanced"` o `scale_pos_weight`) y evaluación sobre el test original desbalanceado. XGBoost domina la comparativa en PR-AUC (0,171 frente a 0,136 de Regresión Logística y 0,124 de Random Forest), F1-Score (0,236 frente a 0,211) y eficiencia computacional (≈ 21 s frente a los $\approx 1,307$ s de la GNN o los ≈ 171 s de la LSTM).
2. **Decisión empírica sobre SMOTE:** Inicialmente se planificó SMOTE como técnica de balanceo. La evaluación sobre el test desbalanceado mostró que SMOTE **degrada** el rendimiento de XGBoost en este dataset (F1 0,19 con SMOTE frente a 0,25 con `scale_pos_weight` optimizado en NB05). La hipótesis, coherente con la naturaleza sintética del *Bank Account Fraud Dataset* [4], es que generar muestras sintéticas sobre un dataset ya sintético introduce ruido y difumina las fronteras de decisión. En consecuencia, la versión de producción descarta SMOTE y adopta `scale_pos_weight`, que modifica los gradientes sin alterar el espacio de *features* observado.
3. **Optimización de hiperparámetros:** Se aplicó *RandomizedSearchCV* con 3-fold CV y 30 iteraciones sobre ocho hiperparámetros de XGBoost, empleando `average_precision` como métrica de *scoring*. La ganancia observada es modesta y realista (F1 0,2437 $\rightarrow$ 0,2490, PR-AUC 0,1744 $\rightarrow$ 0,1830), sin los saltos artificiosos que caracterizan las evaluaciones sobre datos balanceados. La búsqueda exhaustiva sobre Random Forest y Regresión Logística se realizó en NB03 (bajo SMOTE, descartada después) y reportó ganancias negativas que refuerzan la brecha estructural con *gradient boosting*.
4. **Evaluación honesta sobre test original desbalanceado:** El modelo final obtiene F1 $\approx 0,25$, Recall $\approx 0,27$, PR-AUC $\approx 0,18$ y ROC-AUC $\approx 0,90$ sobre el 15 % de test que preserva la prevalencia de fraude del 1,1 %. Estas cifras son las que deben compararse con el ROI del Hito 1: un Recall del 27 % con Precision del 23 %

representa una densidad de señal más de 20 veces superior a la tasa base para los analistas humanos, cumpliendo el objetivo de negocio.

5. **Explicabilidad con SHAP:** El análisis de *summary plot* y *waterfall plot* confirma que las variables comportamentales (`payment_type_*`, `income`, `name_email_similarity`, `bank_months_count`) dominan las decisiones del modelo, ofreciendo trazabilidad suficiente para justificar alertas frente a analistas y clientes.

Lección metodológica principal. La iteración de trabajo ha puesto de manifiesto el riesgo de confiar en métricas cercanas a la perfección cuando el problema de base es extremadamente desbalanceado. Un F1-Score de 0,99 en fraude con prevalencia del 1 % debe considerarse una *señal de alarma* que obliga a revisar el protocolo de evaluación, no un indicador de éxito. La versión corregida del informe reemplaza estas cifras por métricas reales sobre test desbalanceado y asume explícitamente que el valor del sistema reside en la densificación de señal para los analistas, no en una detección perfecta automatizada.

Limitaciones reconocidas. (1) El *split* actual es aleatorio y no temporal; futuros hitos deberán validar con particiones por ventana de tiempo para capturar *concept drift*. (2) No se aplicó búsqueda de hiperparámetros exhaustiva a Random Forest ni a Regresión Logística por restricciones de cómputo; una revisión futura con infraestructura *cloud* debería cerrar ese *gap*. (3) La ingeniería de *features* es manual; queda pendiente explorar técnicas automatizadas (AutoML, interacciones generadas automáticamente) que podrían elevar el PR-AUC. (4) La calibración formal de probabilidades (Hosmer-Lemeshow, *reliability diagrams*) no se ha incluido en este hito y es requisito previo al despliegue en producción.

Capítulo 4

Hito 4: Despliegue del modelo y paso a producción

En los Hitos 1 a 3 se diseñó, validó, entrenó y evaluó un modelo XGBoost que detecta fraude bancario con un PR-AUC de 0,183 y un ROC-AUC de 0,899 sobre el conjunto de test real desbalanceado del *Bank Account Fraud Suite* (NeurIPS 2022). El resultado vive como un artefacto `joblib` dentro de un *notebook* de Jupyter. El presente Hito 4 convierte ese artefacto en un **sistema funcional**: un servicio que recibe transacciones desde fuera, devuelve una decisión de fraude/no fraude en milisegundos y puede reentrenarse con un único comando si los datos cambian.

Todo el código y la documentación de operación viven en la carpeta `deployment/` del repositorio (ver `deployment/README.md` para la guía *plug-and-play*).

4.1. Análisis de las necesidades del sistema

Antes de elegir tecnologías conviene caracterizar la carga de trabajo del servicio. El enunciado plantea tres preguntas clave (slide 5 del Hito 4): ¿requiere GPU?, ¿es intensivo en CPU?, ¿necesitamos almacenar mucho vídeo? Las respuestas para GuardianAI son las siguientes.

GPU. No se requiere. El modelo en producción es un XGBoost con 223 árboles y profundidad máxima 4; la inferencia consiste en recorrer un bosque relativamente pequeño y devolver una probabilidad. La latencia objetivo (decenas de milisegundos por transacción) se cubre sobradamente en CPU. Anexar GPU encarecería la infraestructura sin mejorar el SLA y complicaría el despliegue (drivers, contenedores con CUDA).

CPU. Moderadamente intensivo. La inferencia individual es muy ligera, pero el pico real lo produce el contenedor de *entrenamiento* cuando se ejecuta una búsqueda completa de hiperparámetros (`RandomizedSearchCV` con 30 iteraciones y validación cruzada de 3 *folds* sobre ~ 700.000 filas). Por eso la solución separa entrenamiento e inferencia (Sección 4.4): el contenedor de entrenamiento puede solicitar muchos núcleos cuando se invoca, mientras que el de inferencia se mantiene austero y se replica horizontalmente si hace falta más *throughput*.

Almacenamiento. Modesto. La entrada y la salida son tabulares (un JSON por transacción, ~ 1 KB), no hay vídeo ni binarios pesados. Los artefactos del modelo ocupan menos de 400 KB. El único activo voluminoso es `Base.csv` (213 MB), pero solo lo necesita el contenedor de entrenamiento y de forma puntual, ya que el almacenamiento permanente de transacciones bancarias podría traer problemas legales, de privacidad y de seguridad,

por lo que el dataset, fuera de los periodos de entrenamiento, no sería almacenado en el servidor. Por tanto basta con un volumen local en el host; no se justifica un *object storage* en esta primera infraestructura, aunque la arquitectura está preparada para sustituir el volumen por MinIO o S3 sin cambios en el código de los contenedores (Sección 4.5).

Cargas de trabajo y tipos de cliente. El servicio se diseña para una carga **mixta**: peticiones *streaming* de una transacción (caso del flujo transaccional bancario en tiempo real) y peticiones *batch* de hasta 10.000 transacciones (recomputos nocturnos, *backtesting*, simulaciones del equipo de modelado). Los clientes esperados son sistemas internos del banco (motor antifraude, cuadros de mando del equipo de *ops*); no hay clientes IoT ni móviles que justifiquen protocolos especializados.

4.2. Selección de tecnologías para el despliegue

Contenerización: Docker. Es el estándar *de facto* para distribuir aplicaciones reproducibles. Cada contenedor declara explícitamente sus dependencias y aísla su entorno de Python, lo que elimina la categoría completa de errores “en mi máquina funciona”.

Orquestación local: Docker Compose. El enunciado lo fija como requisito. Es la elección natural para coordinar dos contenedores en un solo host: declara los servicios, sus volúmenes, las variables de entorno y las dependencias entre ellos en un único `docker-compose.yml`. Se descartó Kubernetes para esta primera infraestructura porque introduce complejidad operativa (*cluster*, `kubectl`, manifiestos) que no aporta valor con un único nodo. La migración a Kubernetes sería directa cuando se requiera multinodo (Sección 4.9): cada **service** se transforma en un **Deployment** (inferencia) o un **Job/CronJob** (entrenamiento).

Framework de la API: FastAPI. El enunciado lista cuatro candidatos (slide “Ejemplo VI: API”): Flask, FastAPI, MLrun y KServe. Se eligió FastAPI por tres motivos:

- **Validación declarativa con Pydantic.** Cada transacción tiene 31 campos con tipos, rangos y categorías permitidas. Pydantic los valida automáticamente y devuelve un `422 Unprocessable Entity` con un error legible si el cliente envía algo inválido. En Flask habría que escribir esa validación a mano y mantenerla.
- **OpenAPI / Swagger UI gratis.** El `endpoint /docs` se autogenera a partir de los modelos Pydantic. Cualquier persona puede explorar y probar la API sin documentación adicional.
- **Rendimiento.** FastAPI sobre `uvicorn` (servidor ASGI) ofrece un *throughput* muy superior a Flask con *workers* síncronos, suficiente para las cargas previstas. Si en el futuro se requieren miles de peticiones por segundo, basta con incrementar la variable `UVICORN_WORKERS` o desplegar varias réplicas detrás de un *Ingress*.

Se descartó KServe / Open Inference Protocol por sobreingeniería para el alcance actual (un único modelo, sin *traffic splitting*). La API queda no obstante diseñada para poder convivir, en el futuro, detrás de un servicio compatible con OIP.

Persistencia entre fases: volumen compartido. Se descartaron dos alternativas: (1) *object storage* (S3, MinIO) que añade un servicio extra, *secrets* y latencia de red sin justificación para un solo host, y (2) modelo embebido en la imagen de inferencia (*image-baked*) que obligaría a reconstruir y volver a desplegar la imagen en cada reentrenamiento. El volumen compartido es la solución más *plug-and-play* y, lo más importante, define un contrato (tres ficheros con nombres fijos) que se mantiene si en el futuro se sustituye el *bind mount* por MinIO sin tocar el código.

4.3. Del notebook al sistema: división en módulos

El *notebook* `05_final_model_production.ipynb` concentra todo el flujo en un único fichero: ingesta, exploración, partición, preprocesado, entrenamiento, búsqueda de hiperparámetros, evaluación y persistencia. Este formato es ideal para experimentar pero **no es ejecutable de forma desatendida** y mezcla preocupaciones que en producción deben separarse.

Siguiendo la regla del Hito 4 (slide “Ejemplo II”), el notebook se descompone en cuatro módulos con una única responsabilidad cada uno, ubicados en `deployment/training/pipeline/`:

- `config.py`: centraliza rutas, listas de columnas (continuas, categóricas, *passthrough*), `dtype map` para la carga eficiente de `Base.csv`, semilla aleatoria, parámetros de la búsqueda y los hiperparámetros validados en el Hito 3 (`BEST_PARAMS_HIT03`).
- `data_ingestion.py`: única responsabilidad de I/O hacia la fuente de datos. Lee `Base.csv` aplicando el `dtype map` (reduce ~50 % el consumo de RAM frente al *default*). En producción, este módulo se sustituiría por una conexión a un *data warehouse* o un *bucket* S3 sin tocar el resto del pipeline.
- `features.py`: construye el `ColumnTransformer` (`StandardScaler` para continuas, `OneHotEncoder` para categóricas, *passthrough* para binarias). Este artefacto se serializa con `joblib` para que el contenedor de inferencia lo cargue tal cual y aplique exactamente las mismas transformaciones.
- `train.py`: orquestador end-to-end. Realiza el *split* estratificado 70/15/15, ajusta el preprocesador en `train` (sin *leakage*), calcula `scale_pos_weight` sobre `train`, ejecuta el entrenamiento (modo `quick` o `full`, ver Sección 4.7), busca el umbral óptimo (máximo F1 sobre la curva PR del test), y persiste modelo, preprocesador y metadatos en el volumen compartido.

Análogamente, en `deployment/inference/api/`:

- `settings.py`: rutas de los artefactos y umbrales por variable de entorno.
- `predictor.py`: *wrapper* que carga modelo y preprocesador una sola vez al arrancar (*lifespan* de FastAPI) y expone `predict(...)`. Encapsula también la lógica de selección del umbral activo: variable de entorno `THRESHOLD_OVERRIDE` >campo `umbral_optimo` de `metadatos.json` >`DEFAULT_THRESHOLD`.
- `schemas.py`: contratos Pydantic. Define `Transaccion` (los 31 campos), `LoteTransacciones` (lista de hasta 10.000), y los modelos de respuesta (`PrediccionResponse`, `HealthResponse`, `MetadataResponse`).

- `main.py`: *wiring* HTTP de FastAPI. Define los *endpoints*, el *lifespan* que carga los artefactos al arrancar y el manejo de errores.

Por qué esta división y no otra. Son las fronteras naturales que aparecen al pensar en el ciclo de vida real. La ingesta cambiaría si mañana migramos a S3; el preprocesador cambia si añadimos *features*; la búsqueda de hiperparámetros cambia si subimos a un *cluster*; la API cambia si rediseñamos el contrato. Aislar cada cambio a un fichero permite modificarlo sin tocar el resto.

Lo que deliberadamente no se replicó. El módulo `src/evaluation.py` del repositorio contiene también funciones de *plotting* (curvas ROC y PR, matrices de confusión) que solo tienen sentido en *notebooks*. En `deployment/` solo se replicó la función `buscar_umbral_optimo` que es la única necesaria en producción. La razón: el contenedor debe ser **autocontenido** (no depende de `src/` del repo) y **minimal** (no trae código de exploración).

Configurabilidad por entorno. Siguiendo la regla del Hito 4 (slide “Ejemplo II”), **nin-guna** ruta está *hard-codeada*: `DATASET_PATH`, `ARTIFACTS_DIR`, `REPORTS_DIR`, `MODEL_FILENAME`, `THRESHOLD_OVERRIDE`, `API_PORT`, etc. todas tienen un valor por defecto razonable y pueden sobrescribirse desde `.env` o `docker compose run -e VAR=...`. Esto significa que la misma imagen vale para distintos entornos (desarrollo, *staging*, producción) sin reconstruir.

4.4. Arquitectura de contenedores

La solución se compone de **tres servicios** declarados en `docker-compose.yml`:

1. **bootstrap**: contenedor *one-shot* basado en `busybox:1.36`. Comprueba si el volumen de artefactos está vacío y, si lo está, copia el modelo preentrenado del Hito 3 (carpeta `models/` del repositorio) hacia el volumen compartido. Es la pieza que hace el sistema verdaderamente *plug-and-play*: `docker compose up` funciona en una máquina recién clonada sin necesidad de descargar `Base.csv` ni reentrenar.
2. **training**: contenedor de entrenamiento. Construido sobre `python:3.11-slim` más `libgomp1` (*runtime* OpenMP requerido por XGBoost). Lee `Base.csv` desde un volumen *read-only* y escribe los artefactos al volumen compartido. Está bajo el perfil `train`, por lo que no se levanta automáticamente; solo se invoca cuando hace falta reentrenar (`docker compose -profile train run -rm training`).
3. **inference**: contenedor de la API. Construido sobre la misma imagen base más `curl` para el *healthcheck*. Lee los artefactos del volumen compartido en modo **solo lectura** (`:ro`, evita que un *bug* en la API pueda corromper los artefactos), y expone FastAPI sobre `uvicorn` en el puerto 8000. Declara `depends_on`: `bootstrap` (`service_completed_successfully`) para garantizar que arranca después de la siembra.

Por qué dos contenedores en lugar de uno. El enunciado lo exige (slide “Ejemplo IV”), pero hay además razones técnicas sólidas. Cada contenedor tiene necesidades muy distintas: el de entrenamiento se ejecuta puntualmente con picos altos de RAM y CPU y necesita acceso a `Base.csv`, mientras que el de inferencia es un proceso 24/7 con latencia crítica, RAM moderada y sin acceso a datos en bruto. Empaquetarlos juntos significaría que el contenedor de inferencia (que en producción podría tener decenas de réplicas) cargaría dependencias de entrenamiento que nunca usa. Separarlos da imágenes más pequeñas, despliegues más rápidos y un *blast radius* de fallos más acotado: si el *job* de entrenamiento muere, la API sigue sirviendo el modelo anterior; si la API se cae, no afecta al pipeline de *re-train*. Esta separación es además un prerequisite para implementar el *feedback loop* (slide “Proyecto DISIA: Hito 4”) en futuros hitos.

Optimizaciones de las imágenes. Ambas imágenes usan `python:3.11-slim` (~120 MB) en lugar de la imagen completa (~1 GB). Solo se añade lo imprescindible. `requirements.txt` se copia e instala **antes** del código de la aplicación, de modo que un cambio en un `.py` no invalida la capa de `pip` y los *rebuilds* tardan segundos en lugar de minutos.

4.5. Compartición de datos entre fases

El único contrato entre los contenedores de entrenamiento e inferencia son **tres ficheros con nombres fijos** en una carpeta común (`./shared/artifacts/` en el host, `/artifacts` dentro de los contenedores):

- `modelo_final.joblib`: el clasificador XGBoost entrenado.
- `preprocesador.joblib`: el `ColumnTransformer` ajustado en `train`.
- `metadatos.json`: información del modelo (fecha de entrenamiento, hiperparámetros, métricas *test*, umbral óptimo, listado de columnas continuas/catóricas).

El contenedor de entrenamiento monta el volumen en lectura-escritura; el de inferencia lo monta en solo lectura. Esto evita que un *bug* en la API pueda corromper los artefactos: solo entrenamiento puede escribir.

El patrón *bootstrap*. Sin medidas adicionales, levantar la inferencia en una máquina recién clonada fallaría porque el volumen está vacío. Para evitar forzar al usuario a reentrenar antes de probar, el sistema añade un servicio `bootstrap` que comprueba si los artefactos existen y, en caso contrario, los siembra desde la carpeta `models/` (donde el Hito 3 dejó el modelo preentrenado). Es **idempotente** (si los artefactos ya están, no hace nada), **no invasivo** (no modifica los ficheros originales en `models/`, solo los copia) y **reutilizable** (el mismo patrón sirve en Kubernetes como un `initContainer`).

Estructura final de carpetas.

```
deployment/
  docker-compose.yml      <- orquesta los 3 servicios
  .env.example            <- variables opcionales (copia a .env)
  README.md               <- guia plug-and-play
```

```

training/
  Dockerfile
  requirements.txt
  pipeline/
    config.py
    data_ingestion.py
    features.py
    train.py
inference/
  Dockerfile
  requirements.txt
  api/
    settings.py
    predictor.py
    schemas.py
    main.py
shared/artifacts/          <- volumen compartido
data/
  raw/                     <- aqui va Base.csv si se reentrena
  samples/                 <- JSONs de ejemplo para probar la API

```

4.6. API de inferencia: contrato y endpoints

La API expone los siguientes *endpoints*:

- GET / – redirección a /docs (Swagger UI autogenerada).
- GET /health – diagnóstico. Distingue “vivo” de “vivo y con modelo cargado”. Un orquestador puede usarlo para no enviar tráfico hasta que `modelo_cargado: true`.
- GET /metadata – metadatos del modelo activo (modelo, métricas *test*, fecha de entrenamiento, número de columnas continuas y categóricas). Es la forma de auditar qué modelo se está sirviendo sin abrir el contenedor.
- POST /predict – predicción para una transacción.
- POST /predict/batch – predicción para hasta 10.000 transacciones.

Carga útil de respuesta. Una predicción contiene cinco campos:

```

{
  "probabilidad_fraude": 0.0123,
  "etiqueta": 0,
  "umbral": 0.9177,
  "decision": "PERMITIR",
  "nivel_riesgo": "BAJO"
}

```

Se devuelve el **umbral aplicado** además de la probabilidad cruda. Esto es deliberado: el cliente no necesita conocer el umbral para interpretar la decisión, pero queda en el *log* si mañana hay que auditar por qué se bloqueó una operación. La etiqueta **nivel_riesgo** (BAJO/MEDIO/ALTO/CRITICO) es una abstracción legible para analistas de *ops*, calculada en función de la cercanía de la probabilidad al umbral activo.

Selección del umbral activo. El umbral se carga en el siguiente orden de prioridad: (1) variable de entorno **THRESHOLD_OVERRIDE** (operador en *runtime*), (2) campo **metricas_test.umbral_optimo** de **metadatos.json** (calculado por entrenamiento maximizando F1 sobre la curva PR del test real, en este caso 0,9177), (3) **DEFAULT_THRESHOLD** (*fallback* de seguridad). Esto permite ajustar el *trade-off* precisión/recall sin reentrenar: si negocio decide que prefiere menos falsos positivos basta con **THRESHOLD_OVERRIDE=0.95**
`docker compose restart inference`.

Códigos de error. 422 **Unprocessable Entity** si el JSON no respeta el contrato (lo gestiona Pydantic), 503 **Service Unavailable** si los artefactos no se cargaron correctamente al arrancar, y 400 **Bad Request** si falla el **predict** (preprocesador o modelo).

4.7. Modos de entrenamiento: quick y full

El Hito 3 ya validó qué hiperparámetros son óptimos (almacenados en **metadatos.json**). Lanzar **RandomizedSearchCV** con $n_{iter} = 30$ y CV de 3 *folds* sobre ~ 700.000 filas tarda decenas de minutos y consume RAM significativa, lo cual es inaceptable para una demostración del despliegue. Por eso **train.py** ofrece dos modos seleccionables por variable de entorno o *flag*:

- **SEARCH_MODE=quick** (por defecto): salta la búsqueda y usa directamente los **BEST_PARAMS_HIT03** definidos en **config.py**. Tarda en torno a 1 minuto. Útil para verificar que el *pipeline* de despliegue funciona end-to-end o para reentrenar con datos nuevos manteniendo la estructura del Hito 3.
- **SEARCH_MODE=full**: ejecuta **RandomizedSearchCV** con el mismo espacio de búsqueda que el NB05 del Hito 3. Tarda 30–60 minutos. Útil cuando hay sospecha de *drift* fuerte y conviene revisar la familia de hiperparámetros.

Ambos modos producen el **mismo formato de artefactos**, así que el contenedor de inferencia los consume de forma transparente.

4.8. Flujo de uso *plug-and-play*

El sistema se diseñó para que un usuario nuevo pueda probarlo con tres comandos:

```
cd deployment/  
docker compose up  
curl -X POST http://localhost:8000/predict \  
    -H "Content-Type: application/json" \  
    -d @data/samples/transaccion_legitima.json
```

Lo que ocurre en orden:

1. Docker Compose construye la imagen `guardianai-inference` (la primera vez).
2. El servicio `bootstrap` (*one-shot*) copia los tres artefactos preentrenados desde `../models/` a `./shared/artifacts/`.
3. El servicio `inference` arranca, carga los artefactos en memoria y queda escuchando en `:8000`.
4. El `curl` envía un JSON de ejemplo y recibe la decisión.

Para reentrenar (opcional, requiere `Base.csv` en `deployment/data/raw/`):

```
docker compose --profile train run --rm training
docker compose restart inference
```

4.9. Limitaciones reconocidas y siguientes pasos

Esta es una **primera infraestructura funcional**, lo que pide explícitamente el Hito 4 (slide “¿Qué haréis en el hito 4?”), no una solución productiva completa. Lo que deliberadamente no se incluye y lo que la sustituiría en una iteración posterior:

- **Object storage**: el volumen local no escala a múltiples nodos. Sustituir por **MinIO** o S3 (*bonus* del enunciado) manteniendo el contrato de tres ficheros con nombre fijo. El cambio sería transparente para el código de los contenedores.
- **Tracking de experimentos**: no se versionan modelos ni se registran las corridas de entrenamiento. Añadir **MLflow** (*bonus* del enunciado) sería un cambio aislado en `train.py` (un `mlflow.log_artifact(...)`).
- **Orquestación multinodo**: Compose es para un solo *host*. Migrar a **Kubernetes** (*bonus* del enunciado) es directo: `inference` se convierte en un `Deployment` con `Service`, `training` en un `Job` o `CronJob`, el volumen compartido en un `PersistentVolumeClaim` (o un *bucket* MinIO con S3CSI) y `bootstrap` en un `initContainer`.
- **Autenticación y autorización**: la API está abierta. En producción se añadiría OAuth2/JWT, idealmente delegado a un API *gateway* (Kong, Apigee, Traefik).
- **Detección de drift**: el *monitoring* queda fuera del alcance del Hito 4 (slide “Monitoring and feedback loop”). La separación en dos contenedores ya está preparada para integrarlo como un tercer servicio (Evidently, WhyLogs).
- **Calibración de probabilidades**: el modelo XGBoost no está calibrado formalmente; las probabilidades son útiles para *ranking* pero no para interpretación absoluta. La calibración (`CalibratedClassifierCV`) sería un añadido en `train.py` sin tocar la API.

4.10. Resumen

Qué se ha implementado.

- Dos contenedores Docker (**training** e **inference**) más un *one-shot bootstrap*, orquestados con Docker Compose.
- Una API REST en FastAPI con cinco *endpoints* (`/`, `/health`, `/metadata`, `/predict`, `/predict/batch`), validación Pydantic y documentación interactiva autogenerada.
- División del *notebook* de producción en cuatro módulos con responsabilidad única, manteniendo coherencia con el código de los Hitos previos.
- Compartición de artefactos vía volumen montado en lectura-escritura (entrenamiento) y solo lectura (inferencia), con un contrato explícito de tres ficheros.
- Configurabilidad total por variable de entorno: ninguna ruta *hard-codeada*.
- Documentación de operación (`deployment/README.md`).

Cómo enlaza con los Hitos previos. El contenedor de inferencia consume directamente los artefactos producidos por el NB05 del Hito 3 (`models/modelo_final.joblib`, `models/preprocesador.joblib`, `models/metadatos.json`); las métricas servidas en `/metadata` son las mismas del Hito 3. El contenedor de entrenamiento reproduce el flujo del NB05 fuera del *notebook*, garantizando que se puede regenerar el modelo cuando los datos cambien.

Listo para evolucionar. La arquitectura está preparada para añadir MinIO, MLflow y Kubernetes (*bonus* del enunciado) sin reescribir el núcleo: cada uno sustituye una pieza acotada (volumen, persistencia de experimentos, orquestación) sin tocar el contrato entre entrenamiento e inferencia.

Capítulo 5

Hito 5: Monitorización y *feedback loop*

5.1. Introducción

Un modelo desplegado no es un modelo terminado completamente. Una vez en producción, el sistema se enfrenta a tres riesgos que el conjunto de test no anticipa: por ejemplo, la degradación operativa (latencia, consumo de recursos de la infraestructura que hay por detrás), *data drift* (cambios en la distribución de las solicitudes entrantes respecto a las usadas para entrenar) y degradación de la calidad predictiva (caída de F_1 , AUC o precisión sobre datos reales). El Hito 5 cierra el ciclo de vida del Hito 4 añadiendo una capa de observabilidad y monitoreo que detecta los tres tipos de degradación, un canal de alertas que notifica al equipo y un *feedback loop* que permite reentrenar y desplegar un modelo nuevo sin reiniciar el servicio.

El alcance fijado por el enunciado lo hemos alcanzado con la arquitectura descrita en este capítulo: (i) métricas operativas y de modelo expuestas vía Prometheus, (ii) detección de *data drift* estadístico sobre las solicitudes recibidas, (iii) sistema de alertas por Telegram con al menos una alerta operativa y una de modelo, (iv) simulación de entorno anómalo y (v) *feedback loop* con reentrenamiento simple y recarga en caliente del modelo.

5.2. Selección de tecnologías

Hemos optado por un *stack* de observabilidad ligero, alineado con la simplicidad del despliegue del Hito 4: Prometheus para recolección de métricas y un servicio Python propio (**monitor**) para la lógica de alertas y *drift*. Esta combinación permite que el sistema sea autocontenido (un único **docker compose up**) y se evita la dependencia de plataformas externas (Grafana Cloud, Datadog, etc.). El detalle de cada decisión se resume en la Tabla 5.1.

Pieza	Tecnología elegida	Justificación
Instrumentación de la API	<code>prometheus-fastapi-instrumentator</code> + <code>prometheus_client</code>	Instrumentar <i>de facto</i> en FastAPI. Expone <code>/metrics</code> sin código adicional para las métricas HTTP (peticiones por segundo, latencia, códigos de estado).
Recolector de series temporales	Prometheus 2.51	Pull-based, retención local de 7 días, sin necesidad de servidor externo.
Servicio de alertas y <i>drift</i>	Python 3.11 (<code>psutil</code> , <code>requests</code>)	La lógica de <i>drift</i> estadístico y la deduplicación de alertas son específicas del dominio; un servicio propio mantiene la lógica visible y modificable. Evita la complejidad de <code>Alertmanager</code> para los pocos canales que necesitamos.
Canal de alertas	Bot de Telegram	Sin coste, sin servidor SMTP, sin dependencias corporativas. Trivial de configurar para una demo académica y suficiente para mostrar el patrón “alerta inmediata + recordatorio + recuperación”.
Métricas de modelo	<i>Sliding window</i> en memoria + <code>scikit-learn</code> en la API	Las predicciones reciben un <code>request_id</code> (UUID) y se correlacionan con su etiqueta real diferida vía <code>/feedback</code> . <code>scikit-learn</code> ya está en el contenedor de inferencia.

Tabla 5.1: Tecnologías seleccionadas para el Hito 5.

5.3. Arquitectura de observabilidad

Hemos ampliado el despliegue del Hito 4 se amplía con dos contenedores nuevos (`prometheus` y `monitor`) y hemos modificado el contenedor `inference` para que exponga métricas y registre cada predicción. El resto de servicios (`bootstrap`, `training`) los hemos podido mantener iguales, ya que la arquitectura del Hito 4 estaba lista para evolucionar como se anticipaba en la Sección 4.9.

- `inference` expone `/metrics` en formato Prometheus con (a) métricas HTTP autoinstrumentadas (peticiones por segundo, latencia por *endpoint*, códigos de estado), (b) *gauges* propios de sistema (`guardianai_cpu_percent`, `guardianai_ram_percent`), (c) métricas de negocio (`guardianai_fraud_rate`, `guardianai_predictions_total`) y (d) métricas de calidad del modelo confirmadas (`guardianai_model_f1`, `_auc`, `_precision`, `_recall`, `_confirmados`).
- Cada llamada a `/predict` y `/predict/batch` se registra como una línea JSON en `shared/logs/predictions_log.jsonl` con *todas* las features de entrada, el `request_id` asignado, la probabilidad, la etiqueta predicha y la decisión. Este *log* alimenta el cálculo de *drift* en el contenedor `monitor`.

- **prometheus** hace *scraping* de **inference** cada 15 segundos y conserva las series 7 días localmente.
- **monitor** es un servicio Python en bucle infinito (cada `CHECK_INTERVAL=60s` por defecto) que mide CPU/RAM, lee el *log* de predicciones, consulta `/model-metrics` y decide si emite alertas por Telegram. Tiene montado `/var/run/docker.sock` para poder disparar reentrenamientos cuando `RETRAIN_ON_DRIFT=true`.

5.4. Métricas operativas

Hemos conseguido cubrir los dos requisitos del enunciado:

- **Recursos del sistema:** CPU y RAM del contenedor de inferencia, medidos con `psutil` y publicados como *gauges* Prometheus. El contenedor **monitor** los relee cada ciclo para alertar cuando superan los umbrales (`CPU_THRESHOLD=80 %` y `RAM_THRESHOLD=80 %` por defecto).
- **Tráfico HTTP:** `prometheus-fastapi-instrumentator` expone automáticamente `http_requests_total` (contador por *endpoint* y código de estado) y `http_request_duration_seconds` (histograma de latencias). Cualquier *query* PromQL puede derivar peticiones por segundo (`rate(...[1m])`), peticiones por minuto, latencia p50/p95/p99, etc., sin código adicional.

5.5. Detección de *data drift*

El monitor implementa una detección de *drift* estadístico simple y robusta sobre las diez *features* numéricas más relevantes del modelo final (`income`, `name_email_similarity`, `customer_age`, `days_since_request`, `velocity_6h/24h/4w`, `zip_count_4w`, `proposed_credit_limit` y `session_length_in_minutes`). Para cada una se calcula el *z-score* de la media reciente respecto de las estadísticas de referencia del conjunto de entrenamiento:

$$z_f = \frac{|\bar{x}_f^{\text{reciente}} - \mu_f^{\text{ref}}|}{\sigma_f^{\text{ref}}}.$$

Si $z_f > 3$ en cualquier *feature* (umbral configurable vía `DRIFT_Z_THRESHOLD`), se considera que esa variable presenta *drift*. El cálculo se activa cuando hay al menos `MIN_PREDICTIONS_DRIFT=30` predicciones en la ventana, evitando alarmas en el arranque. Las estadísticas de referencia se cargan desde `shared/artifacts/reference_stats.json` si existen, con un *fallback* a valores codificados en el monitor para preservar el comportamiento *plug-and-play*.

Esta estrategia es deliberadamente más conservadora que un test estadístico completo (Kolmogórov-Smirnov, PSI), sin embargo, hemos considerado que era mejor alternativa para nuestro caso de uso: el coste es baja sensibilidad a cambios de forma sin cambio de media, pero la ventaja es no necesitar guardar las distribuciones de referencia completas y producir un mensaje de alerta autodescriptivo (“`velocity_6h: media=14500.2 (ref=4500, z=3.6)`”).

5.6. Métricas del modelo y *ground truth* diferido

El enunciado pide exponer métricas clásicas de modelo (F_1 , AUC, precisión). En un escenario real de detección de fraude estas métricas son **imposibles de calcular en tiempo real**: cuando la API decide PERMITIR o BLOQUEAR una transacción no se conoce la etiqueta verdadera, ya que llega más tarde, a través de reclamaciones del cliente o *chargebacks*, con latencias que oscilan entre horas y semanas. Pensamos que cualquier sistema de monitorización honesto debe asumir esta brecha temporal.

La solución implementada modela explícitamente esa brecha con tres piezas:

1. **Identificación de la predicción.** Cada llamada a `/predict` devuelve un `request_id` (UUID) además de la decisión. El `request_id` se almacena en una caché en memoria (`ModelMetricsTracker`) junto con la probabilidad y la etiqueta predicha.
2. **Recepción del *ground truth*.** Un nuevo *endpoint* POST `/feedback` acepta pares `{request_id, fraud_bool_real}` y los correlaciona con la predicción correspondiente. En producción real este *endpoint* sería llamado por el sistema de *back office* bancario cuando se confirma una reclamación; en el sistema demostrador lo usa el simulador de anomalías (Sección 5.8).
3. **Ventana móvil de métricas.** Cuando un *feedback* se correlaciona, se añade el triplete (probabilidad, etiqueta predicha, etiqueta real) a una ventana de las últimas $N = 200$ predicciones confirmadas. Se recalculan F_1 , precisión, *recall* y AUC con `scikit-learn` y se publican como *gauges* Prometheus y vía GET `/model-metrics`.

La ventana móvil tiene dos propiedades deseables: (a) reacciona en tiempo casi real a cambios de calidad sin perder muestras pasadas (estabilidad estadística), y (b) permite alertar cuando F_1 cae por debajo de un umbral configurable (`MODEL_F1_THRESHOLD=0.20` por defecto). El umbral 0,20 se ha elegido por debajo del $F_1 = 0,249$ obtenido en el conjunto de test del Hito 3 (Sección 3.6), de modo que sólo dispare ante degradación significativa del modelo en producción.

Es importante reconocer las limitaciones de este diseño: si el *feedback* es sistemáticamente sesgado (por ejemplo, sólo se reciben reclamaciones de transacciones bloqueadas erróneamente, pero no de fraudes que pasaron como legítimos), las métricas en la ventana móvil estarán igualmente sesgadas. Podríamos por ejemplo resolverlo en producción combinando el *feedback* explícito con auditorías periódicas de muestras aleatorias, lo cual queda fuera del alcance del Hito 5.

5.7. Sistema de alertas y deduplicación

El monitor envía mensajes por Telegram cumpliendo un patrón habitual en sistemas de *paging*:

- **Primera detección → alerta inmediata.** En el siguiente ciclo a la aparición del problema.
- **Mismo problema persiste → recordatorio.** Tras `ALERT_REMINDER_HOURS=1` sin que el problema desaparezca, se reenvía la alerta marcada como recordatorio.

- **Problema resuelto** → **notificación de recuperación**. Una sola vez cuando todas las categorías vuelven al estado limpio.
- **No *spam***. El estado de las cinco categorías (`cpu`, `ram`, `drift`, `fraud_rate`, `model_f1`) se persiste en `shared/logs/alert_state.json`; sólo se envía mensaje cuando el estado *cambia* o cuando expira el recordatorio.

Las cinco categorías cubren el requisito del enunciado de tener al menos una alerta para métrica operativa y una para métrica de modelo: CPU y RAM son operativas; *drift*, tasa de fraude y F_1 son de modelo, cada una de naturaleza distinta (estadística de entrada, distribución de salida y calidad sobre verdad confirmada).

5.8. Simulación de entorno anómalo

El *script* `deployment/simulate_anomaly.py` envía ráfagas de transacciones con *features* típicas de fraude masivo (`velocity_6h=15 000`, `foreign_request=1`, `session_length=0.1 min`, etc.) y dispara las alertas en menos de un ciclo del monitor. Con la bandera `-feedback` el simulador genera, en hilos independientes, el envío diferido de `/feedback` con la etiqueta real (1 si la plantilla era de fraude, 0 si era legítima), con retrasos aleatorios en `[feedback_delay_min, feedback_delay_max]` segundos. Este patrón reproduce, a escala temporal comprimida, el flujo real de confirmación de *chargebacks*.

Un ejemplo que podríamos utilizar en una demo sería el siguiente:

```
python simulate_anomaly.py --n 100 --modo mixto --feedback \
--feedback-delay-min 2 --feedback-delay-max 8
```

Tras la ejecución pueden consultarse las métricas resultantes en `GET /model-metrics` o directamente en Prometheus (`guardianai_model_f1`, etc.).

5.9. *Feedback loop*: reentrenamiento y recarga en caliente

El bucle se cierra con dos piezas. La primera es el reentrenamiento: si `RETRAIN_ON_DRIFT=true` y se detecta *drift*, el monitor invoca por el socket de Docker el contenedor `training`, que regenera `modelo_final.joblib`, `preprocesador.joblib` y `metadatos.json` en el volumen compartido. Este reentrenamiento es *simple* (mismo *pipeline* del NB05, sin búsqueda de hiperparámetros salvo que utilizemos el parámetro que hemos dejado para forzarlo `SEARCH_MODE=full`).

La segunda es la recarga en caliente. Sin ella, el modelo nuevo permanecería en disco hasta que un humano reiniciase el contenedor. Para evitarlo se ha añadido el *endpoint* `POST /reload` en la API, que reinstancia el `FraudPredictor` (recargando los tres artefactos) y reinicia la ventana móvil de métricas del modelo. Tras un reentrenamiento exitoso, el monitor llama automáticamente a `/reload` y publica una notificación con la nueva versión y el umbral activo. Esto convierte el *loop* en autosostenido: detección → reentrenamiento → publicación → vuelta al estado de monitorización sin intervención humana.

Por seguridad, `RETRAIN_ON_DRIFT` está desactivado por defecto: habilitarlo implica que el contenedor monitor tiene capacidad para lanzar contenedores en el *host*, un privilegio que debe estar restringido a entornos de demo.

5.10. Limitaciones reconocidas y siguientes pasos

- **Visualización.** No se incluye un *dashboard* Grafana. Las series están en Prometheus y se consultan vía PromQL; añadir Grafana sería un contenedor adicional sin código nuevo.
- **Persistencia de las métricas del modelo.** La ventana móvil vive en memoria; un `docker compose restart` la borra. El `feedback_log.jsonl` sí persiste y bastaría para reconstruirla, pero esa reconstrucción al arranque queda fuera de alcance.
- **Reference *stats* dinámicas.** El entrenamiento no genera todavía `reference_stats.json`; el monitor usa los valores por defecto. Producirlo desde el *pipeline* sería una mejora mínima.
- **Privacidad y normativa.** El *log* de predicciones contiene las 30 *features* de cada solicitud. En producción real esto colisionaría con GDPR/PSD2; un sistema productivo guardaría sólo hashes o agregados a nivel de ventana, no transacciones individuales.
- **Estrategias avanzadas de despliegue.** No se ha implementado A/B testing ni *champion-challenger*. El reentrenamiento promueve directamente el modelo nuevo sin comparar contra el anterior; un siguiente paso natural es servir ambos en paralelo (*shadow testing*) y promover sólo si las métricas del retador superan las del campeón sobre la misma ventana de *feedback*. La pieza que falta para esto es la convivencia de dos predictores en `app.state`; el resto de la infraestructura ya está preparada.

5.11. Resumen

Qué se ha implementado.

- Dos contenedores nuevos (`prometheus`, `monitor`) integrados con los del Hito 4.
- Instrumentación de la API con métricas HTTP automáticas y cinco *gauges* propios de modelo (`f1`, `auc`, `precision`, `recall`, `confirmados`).
- Detección de *data drift* por *z-score* sobre diez *features* numéricas.
- Tres nuevos *endpoints*: `/feedback`, `/model-metrics` y `/reload`.
- Caché en memoria `ModelMetricsTracker` para correlacionar predicciones con su *ground truth* diferido por `request_id`.
- Sistema de alertas Telegram con deduplicación y recordatorios.
- Simulador `simulate_anomaly.py` extendido con *feedback* diferido en hilos.
- *Feedback loop* completo: `drift` → reentrenamiento → recarga en caliente vía `POST /reload`.

Cómo enlaza con los Hitos previos. El Hito 5 se monta sobre la API del Hito 4 sin reescribir ningún módulo de éste: el `FraudPredictor`, el contrato Pydantic de las transacciones y el *pipeline* de entrenamiento siguen siendo los del NB05. Las nuevas piezas se acoplan por composición: un *tracker* en `app.state`, tres *endpoints* adicionales y dos contenedores satélite. Eso confirma que la separación modular elegida en el Hito 4 era apropiada para evolucionar.

Listo para evolucionar. El siguiente paso natural es servir dos modelos en paralelo (*champion-challenger/shadow*) y promover automáticamente el retador cuando supere al campeón en la ventana de *feedback*. La infraestructura ya soporta todo lo necesario: las métricas comparables, el canal de alertas y la recarga en caliente.

Bibliografía

- [1] The Nilson Report (2023). *Global Card Fraud Losses*. Recuperado de la publicación anual sobre métricas de la industria de pagos.
- [2] Baesens, B., Van Vlasselaer, V., & Verbeke, W. (2015). *Fraud Analytics Using Descriptive, Predictive, and Social Network Techniques: A Guide to Data Science for Fraud Detection*. John Wiley & Sons.
- [3] Dal Pozzolo, A., Boracchi, G., Caelen, O., Alippi, C., & Bontempi, G. (2017). *Credit Card Fraud Detection: A Realistic Modeling and a Novel Learning Strategy*. IEEE Transactions on Neural Networks and Learning Systems, 29(8), 3784-3797.
- [4] Jesus, S., Pombal, J., Alves, D., Cruz, A., Saleiro, P., Bizarro, P., & Gama, J. (2022). *Turning the Tables: Biased, Imbalanced, Dynamic Tabular Datasets for ML Evaluation*. Advances in Neural Information Processing Systems (NeurIPS).
- [5] Alnasser, M. (2024). *Machine Learning Based Fraud Risk Scoring for Financial Transactions Using a Comparative Study of Random Forest and Support Vector Regression Models*. ResearchGate. DOI: 10.13140/RG.2.2.21657.
- [6] Chen, T., & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. Proceedings of the 22nd ACM SIGKDD, 785–794.
- [7] Das, S. R., Bin Sulaiman, R., & Butt, U. (2023). *Comparative Analysis of Machine Learning Algorithms for Credit Card Fraud Detection*. FMDB Transactions on Sustainable Computing Systems, 1(4), 225–244.
- [8] Phua, C., Lee, V., Smith, K., & Gayler, R. (2010). *A comprehensive survey of data mining-based fraud detection research*. Artificial Intelligence Review, 34(1), 1–14.
- [9] Oladele, S., John, C., & Marion, L. (2023). *A Comparative Study of Graph Neural Networks (GNNs) vs. traditional supervised learning methods for Identifying Syndicated Credit Fraud and Anomalous Loan Applications*. ResearchGate.
- [10] Tian, X., Li, Y., & Wang, Z. (2024). *Graph neural networks for fraud detection in financial transactions: methods, challenges and opportunities*. IEEE Access.
- [11] Forough, J., & Momtazi, S. (2021). *Ensemble of deep sequential models for credit card fraud detection*. Applied Soft Computing, 99, 106883.
- [12] Hafez, I. Y., Alam, M., & Idris, M. (2025). *A systematic review of AI-enhanced techniques in credit card fraud detection*. Journal of Big Data, 12(1), 1-23.
- [13] Stephen Eteng, et al. (2024). *Generative Adversarial Networks (GANs) for Synthetic Fraudulent Transaction Data Augmentation*. ResearchGate.
- [14] Haiquan Yang, et al. (2026). *A Review of Artificial Intelligence for Financial Fraud Detection*. Applied Sciences, MDPI.

- [15] R. Renuga Devi, et al. (2025). *Reinforcement learning with graph neural network (RL-GNN) fusion for real-time financial fraud detection: a context-aware community mining approach*. Scientific Reports (PMC).
- [16] Yisong Chen, et al. (2025). *Year-over-Year Developments in Financial Fraud Detection via Deep Learning: A Systematic Literature Review*. arXiv.
- [17] KPMG International. (2025). *Bridging innovation and compliance - Machine learning models in Financial Crime Compliance (FCC)*.
- [18] Saito, T., & Rehmsmeier, M. (2015). *The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets*. PLoS ONE, 10(3).
- [19] Martini, A. (2025). *Deep Recurrent Neural Networks for Fraud Detection on Debit Card Transactions*. Barclays Quantitative Analytics.